



**KTO KARATAY
ÜNİVERSİTESİ**

**KTO KARATAY UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING**

SENIOR DESIGN PROJECT - 2

İsim ve Soyisim : Yasin Derya Bilgin

Öğrenci Numarası : 231450075

Proje İsmi : RIMA - Rift Avcıları

Proje Deposu : <https://github.com/ydbilgin/RIMA>

İçindekiler

ÖZET	5
1 PROJE GENEL BAKIŞ	6
1.1 Proje Tanımı	6
1.2 Amaç ve Önem	6
1.3 Kullanılan Teknolojiler	6
1.4 Proje Yapısı	7
2 SİSTEM MİMARİSİ	8
2.1 Bileşen-Tabanlı ve Veri-Güdümlü Mimari	8
2.2 Proje Klasör Yapısı ve Sınıf Sorumlulukları	8
2.3 Sahne Akışı ve Manager Katmanı	9
2.4 Veri → Sahne Boru Hattı	9
3 VERİ MODELLERİ (ScriptableObject Tasarımı)	11
4 ANA SİSTEMLER VE ÖZELLİKLER	12
4.1 Tam Oynanabilir Döngü	12
Attunement Chamber — Yürünebilir Karakter Seçimi	12
Editör Deneme Alanı (Chamber [K])	12
Dövüş (Combat)	12
3-Kart Yetenek Seçimi (Skill Draft)	12
Dallanan Rift Portalları	13
Boss Karşılaşması	13
Zafer / Ölüm ve Meta-İlerleme	13
4.2 Build Sistemi: Skill Draft, Tier Kilitleri ve Sinerji	13
4.3 Sınıf Tasarımı: Warblade Örneği	14
4.4 Düşman AI, Dalga Sistemi ve Boss Telegraph	15
4.5 Oda Akışı State Machine	16
4.6 Oyun Hissi (Game-Feel) Katmanı	16
5 OYUN-İÇİ GELİŞTİRME ARAÇLARI	18
5.1 Build Mode (F2) — Edit-to-Play	18
5.2 Director Mode — Runtime UI Factory	18
5.3 F1 Debug Paneli	19

6 VERİ-GÜDÜMLÜ ODA SİSTEMİ VE ODA İNŞA HATTI.....	20
6.1 Veriden Odaya: Sahneden Veriye ve Otomatik Uçurum Yerleşimi	20
6.2 Otomatik Prop Yerleşimi	21
6.3 Dış Kaynaklı İçerik ve JSON İçerik Aktarma.....	21
6.4 Authored Portal Soketi Sistemi	22
6.5 Geliştirme Araç Zinciri (Editör)	22
7 GÖRSEL ÜRETİM HATTI.....	23
7.1 Sanat Yönetimi.....	23
7.2 Karakter Sprite Üretimi: 10 Sınıf × 8 Yön.....	23
7.3 Piksel-Art İçerik Aktarma Disiplini	23
7.4 Silah Mount Sistemi.....	24
7.5 Ortam Asset'leri ve İki Araç Ayrımı.....	24
7.6 Demo Düşman Kadrosu	24
8 YAPAY ZEKÂ DESTEKLİ ÇOK-AJANLI GELİŞTİRME METODOLOJİSİ.....	26
8.1 Gerekçe: Kaynak Kısıtları	26
8.2 Sanal Ekip Yapısı	26
8.3 Süreç Kuralları	26
8.4 Paralel Görev Kuyruğu.....	27
8.5 Metodolojinin Değerlendirilmesi ve Sınırları.....	27
8.6 İnsan-YZ İş Bölümü ve Çalışma Prensipleri	27
Orkestrasyon Katmanı	27
Ajan Rollerini	28
Temel Disiplinler	28
Mühendislik Çıktısına Yansıması.....	28
9 DOĞRULAMA: TEST VE KALİTE GÜVENCESİ.....	29
9.1 Tek Geliştiricinin Kalite Sorunu.....	29
9.2 Test Altyapısı	29
9.3 Görsel Oda Kalite Güvence Süreci	29
9.4 Üçlü Kalite Güvencesi	30
10 GRAPHIFY KOD-GRAFI AUDİT	31
10.1 Yöntem	31
10.2 God-Node Analizi: Araç Katmanı Tezinin Sayısal Kanıtı.....	31

10.3 Tam Graf Görünümü	32
11 SONUÇ VE GELECEK GELİŞTİRMELER.....	34
11.1 Ulaşılan Yer.....	34
11.2 Yol Haritası.....	34
11.3 Çıkarılan Dersler	34
11.4 Değerlendirme.....	35
11.5 Tasarım Kararları ve Geliştirme İlkeleri	35
EK A — ELEMENTAL DURUM EFEKTLERİ	36
Kaynakça.....	37

ÖZET

Bu çalışma, tek geliştirici tarafından yapay zekâ destekli çok-ajanlı bir geliştirme süreci ve veri-güdümlü oyun mimarisi kullanılarak hayata geçirilen 2D izometrik bir aksiyon-roguelite oyunu olan RIMA'yı sunmaktadır. Projenin temel sorusu şudur: tek kişilik bir ekip, kapsamlı bir roguelite oyununu nasıl üretebilir?

Bunu mümkün kılmak için iki paralel yaklaşım izlenmiştir. İlki, içerik ve sistemin birbirinden ayrıldığı veri-güdümlü bir mimaridir; oyun odaları ScriptableObject veri dosyaları olarak tanımlanmış, zemin-uçurum-prop yerleşimi çalışma zamanında ve otomatik olarak inşa edilmiş, JSON tabanlı içe aktarma araçlarıyla dış kaynaklı oda tasarımları sisteme entegre edilmiştir. İkincisi, kod üretimi, tasarım danışmanlığı ve çıktı incelemesini farklı rollere dağıtan çok-ajanlı bir yazılım mühendisliği sürecidir. Bu süreçte yazar-reviewer ayrımı, karar dökümanı zorunluluğu ve doğrulama kanıtı gerekliliği temel ilkeler olarak uygulanmıştır.

Proje; on sınıflık veri mimarisi (ikisi —Warblade ve Elementalist— uçtan uca oynanabilir demo sınıfı), 26 oda şablonu, 549 otomatik test tanımı (son koşu: 410 geçti / 0 başarısız / 1 belirsiz) ve MainMenu'den boss karşılaşmasına uzanan oynanabilir tam döngüyle sonuçlanmıştır. Test altyapısı EditMode ve PlayMode testlerine ek olarak programatik bir görsel oda kalite güvencesi sürecini kapsar.

Anahtar kelimeler: roguelite, prosedürel içerik, veri-güdümlü tasarım, çok-ajanlı yapay zekâ, oyun geliştirme

1 PROJE GENEL BAKIŞ

1.1 Proje Tanımı

RIMA (Rift Avcıları), tek bir geliştirici tarafından yapay zekâ destekli çok-ajanlı bir geliştirme süreci ve veri-güdümlü bir oyun mimarisi kullanılarak hayata geçirilen 2D izometrik bir aksiyon-roguelite oyunudur. Oyuncu, kendine özgü güçlere ve birbirinden tamamen farklı kaynak ekonomilerine sahip savaşçı sınıflarından birini seçerek prosedürel olarak ilerleyen bir zindan koridorunda oda oda dövüşür; her geçişte yeni yetenekler edinerek daha güçlü bir yapı (build) inşa eder. Başarısız bir koşu (run) dahi sonraki oturumlar için kalıcı bir kazanım bırakır.

Projenin akademik tezi tek başına "bir oyun yapmak" değildir. RIMA, kasıtlı olarak bir **environment + dikey dilim (vertical slice) + yeniden-kullanılabilir oyun-içi geliştirici araç zinciri** olarak konumlanmıştır. Bu çerçevede değerlendirme eksenini içerik hacmine değil, sistem mimarisine ve mühendislik disiplinine ağırlık verir. Oyunun oynanabilir döngüsü tezin kanıtıdır; asıl katkı ise bu döngüyü mümkün kılan veri-güdümlü mimari, oyun-içi araç katmanı ve doğrulanabilir AI-destekli geliştirme sürecidir.

Projenin yanıtladığı temel soru şudur: tek kişilik bir ekip, ticari ölçekli örneklerin (Hades, Dead Cells, Slay the Spire) kapsamını birebir hedeflemeden, aynı türün temel döngülerini veri-güdümlü ve doğrulanabilir biçimde nasıl üretebilir?

1.2 Amaç ve Önem

Aksiyon-roguelite türü, savaş mekaniği, oda çeşitliliği, görsel dil, build derinliği ve meta-ilerleme ekonomisini bir arada gerektirir; bunların her biri kendi başına bir uzmanlık alanıdır. Ticari bir roguelite normalde programcı, seviye tasarımcısı, sanatçı, QA uzmanı ve sistem mimarından oluşan bir ekip ister. Tek kişilik bir projede bu rollerin tümü aynı kişiye düşer ve hem zaman hem de bağımsız gözden geçirme kapasitesi sınırlı kalır.

Bu kaynak kısıtı karşısında iki yol vardır: kapsamı kesmek ya da mevcut kapasiteyi genişletecek bir iş akışı kurmak. RIMA ikinci yolu seçer ve bunu iki paralel mimari kararla yapar. Birincisi, içerik ile sistemi birbirinden ayıran veri-güdümlü mimaridir: her oda ayrı bir sahne değil, bir ScriptableObject veri dosyasıdır; her yetenek merkezi bir veritabanında tanımlanır. İkincisi, kod üretimi, tasarım danışmanlığı ve çıktı incelemesini farklı rollere dağıtan çok-ajanlı bir yazılım mühendisliği sürecidir. Projenin mühendislik önemi, büyük dil modellerini tekil bir "komut ver-çalıştır" aracı olarak değil; rol bazlı, doğrulama döngülü ve kalıcı bir bilgi tabanına sahip etmen-temelli (agentic) bir geliştirme sistemi olarak konumlandırmasında yatar.

1.3 Kullanılan Teknolojiler

Projenin teknoloji yığını, piksel-sanatı izometrik bir oyunun gereksinimleri ve tek-geliştirici üretkenliği gözetilerek seçilmiştir.

- **Oyun motoru:** Unity 6 — proje başladığı dönemde uzun dönemli destek (LTS) aşamasına yaklaşan, API kararlılığı yüksek bir sürüm; tek geliştirici için sürüm-kırılmalarıyla uğraşmak yerine üretkenliği korumak önceliklidir.
- **Render hattı:** Universal Render Pipeline (URP) 2D — pixel-perfect kamera, Light2D dinamik ışık sistemi ve izometrik Tilemap entegrasyonu için.

- **Programlama dili:** C# — Unity'nin birincil betik dili; bileşen-tabanlı (component-based) MonoBehaviour mimarisi.
- **Veri katmanı:** ScriptableObject — odalar, yetenekler, prop tanımları ve silah verisi için kod-dışı, sürümlenebilir veri varlıkları.
- **Giriş sistemi:** Unity Input System — WASD hareket, LMB/RMB saldırı ve Q/E/R/F yetenek tuşları.
- **Görsel üretim hattı:** Pixellab AI (karakter ve VFX sprite'ları, 8-yön), Imagen tabanlı agy boru hattı (ortam ve UI görselleri).
- **AI-destekli geliştirme katmanı:** Çok-ajanlı orkestrasyon (Kod Üretim Modülü ve Mimari Çapraz-Doğrulama kanalları¹), Model Context Protocol (MCP) üzerinden Unity Editor ve Pixellab sürümü, kalıcı bilgi tabanı için NotebookLM, kod-grafı analizi için graphify.

1.4 Proje Yapısı

Projenin tüm kaynak kodu, oda veri varlıkları ve bu rapor açık erişimle GitHub deposunda yayımlanmıştır: <https://github.com/ydbilgin/RIMA>

Tüm oyun kodu Assets/Scripts/ altında toplanmıştır: 625 C# dosyası, 37 alt-dizin. Klasör ağacı sistem sorumluluklarına göre ayrılmıştır; aşağıdaki tablo en büyük dizinleri dosya sayısı ile birlikte özetler:

Dizin	.cs	İçerik
MapDesigner/	98	Oda sistemi, IsoRoomBuilder, RoomRunDirector, template/data
Skills/	92	Yetenek/kart sistemi, DraftManager, cross-class
Systems/	52	Çekirdek oyun sistemleri
UI/	47	BuildMode, DirectorMode, HUD, tooltip
Combat/	45	Dövüş, hasar, stat, juice
Core/	34	RewardPickup, çekirdek akış
Environment/	32	Çevre, ışık, prop
Enemies/	32	Düşman AI/spawn
Runtime/	26	Runtime orchestration
Editor/	25	Unity editör araçları (RoomJsonImporter vb.)

Bu yapı, "içerik ile sistemin ayrılması" ilkesinin fiziksel yansımasıdır: oyun mekaniğini (Skills/, Combat/, Enemies/) tanımlayan kod ile içerik üreten araçları (MapDesigner/, Editor/) tanımlayan kod ayrı katmanlardadır. Geliştirme araçları, kod tabanının yaklaşık beşte birini oluşturur; bu, projenin "geliştirme ortamı" (tooling environment) tezini kod organizasyonu düzeyinde de doğrular.

2 SİSTEM MİMARİSİ

2.1 Bileşen-Tabanlı ve Veri-Güdümlü Mimari

RİMA, Unity'nin bileşen-tabanlı (component-based) modelini veri-güdümlü (data-driven) bir tasarım disipliniyle birleştirir. Temel ilke şudur: davranış kodda, içerik veride yaşar. Bir oda nasıl inşa edileceğini bilen kod (IsoRoomBuilder) ile o odanın ne olduğunu tanımlayan veri (RoomTemplateSO) birbirinden ayrıdır. Aynı ayırım yetenekler (SkillData ↔ SkillController), prop'lar (PropDefinitionSO ↔ BridsonPoissonAutoPlacer) ve silahlar (WeaponDatabaseSO ↔ HandAnchorAttach) için de geçerlidir.

Bu ayırımın pratik sonucu, içerik ölçeklenmesinin maliyetini düşürmesidir: yeni bir oda eklemek yeni bir sahne kurmak değil, yeni bir veri varlığı oluşturmaktır; yeni bir yetenek eklemek veritabanına bir kayıt girmektir.

2.2 Proje Klasör Yapısı ve Sınıf Sorumlulukları

Tüm oyun kodu Assets/Scripts/ altında, sistem sorumluluklarına göre ayrılmış modüler bir klasör ağacında toplanmıştır. Aşağıdaki ağaç, kod tabanının küratörlü bir görünümüdür; her modül tek bir mimari katmana karşılık gelir:

Assets/Scripts/	
MapDesigner/	oda sistemi: IsoRoomBuilder, RoomRunDirector, RoomTemplateSO, prop yerleşimi
Skills/	yetenek/kart sistemi: SkillBase, sınıf SkillController'ları,
DraftManager,	cross-class
Systems/	çekirdek oyun sistemleri ve singleton yöneticiler
Combat/	dövüş, hasar, stat, game-feel (juice) sürücüleri
UI/	BuildModeController, DirectorMode, HUD, SkillBarUI, tooltip
Core/	RewardPickup ve çekirdek run akışı
Environment/	çevre, Light2D ışık, prop tanımları
Enemies/	düşman AI (BaseMobBehavior) ve dalga/spawn
Runtime/	runtime orkestrasyon ve kayıt (registry) baker'ları
Editor/	Unity editör araçları (RoomJsonImporter, Map Designer vb.)
Data/ Player/ Map/ VFX/ Audio/	veri varlıkları, oyuncu, harita yardımcıları, efekt, ses

Kod-graflı analizinde (bkz. Bölüm 10) en yüksek bağlantı derecesine sahip "god-node" sınıflar, sistemin etrafında döndüğü merkezlerdir. Bu sınıfların sorumlulukları ve etkileşimleri şu hiyerarşiyle özetlenebilir:

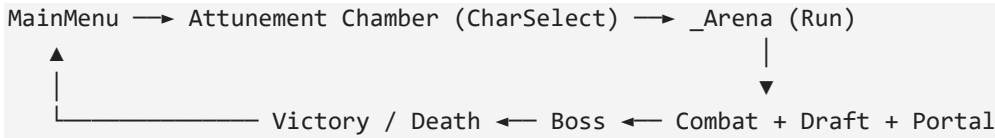
- **`RoomRunDirector`** (oda yaşam-döngüsü orkestratörü). Bir run boyunca odaların yaşam döngüsünü (build → combat → cleared → reward → door → advancing) yedi durumlu bir durum makinesiyle yönetir. IsoRoomBuilder'a hangi şablonun kurulacağını söyler, EncounterController'dan "temizlendi" sinyalini bekler, ödül ve portal akışını tetikler; üst düzeyde DungeonGraph'tan gelen run-map'e göre bir sonraki odayı seçer. Oyun-tarafının merkezî koordinatörüdür.
- **`IsoRoomBuilder`** (veri → sahne inşa motoru). Salt-veri olan RoomTemplateSO'yu okuyup zemin karolarını, otomatik uçurum (cliff) sprite'larını ve bake edilmiş prop'ları tek bir _Arena sahnesine yerleştirir. RoomRunDirector tarafından her oda geçişinde çağrılır; veri katmanı ile çalışan sahne arasındaki köprüdür.

- **`SkillController` (sınıf-bazlı aile).** Yetenek davranışı, ortak bir SkillBase soyut sınıfı ve her oyuncu sınıfı için ayrı bir controller (Warblade_SkillController, Elementalist_SkillController, Ranger_SkillController, Shadowblade_SkillController) ile gerçekleştirilir. Controller, SkillData veri kayıtlarını çalışma-zamanı eylemine bağlar; slot durumu SkillBarUI ile, draft akışı DraftManager ile etkileşir. Bu, "imza yetenekler bespoke kalır" tasarım disiplininin kod düzeyindeki karşılığıdır.
- **`BuildModeController` ve `DirectorMode`** (oyun-içi araç god-node'ları). DirectorMode çalışma zamanında sekmeli bir UI üreten geliştirici konsoludur (Stat/Spawn/Telemetry/Build/Map); kod-grafında en bağlı node'dur, çünkü hemen her sisteme canlı bir müdahale arayüzü sağlar. BuildModeController, DirectorMode'un toggle altyapısı üzerine oturan oyun-içi seviye editörünü (F2 / ") yönetir ve prop yerleştirmeyi BuildPlacementController'a devreder. Her iki sınıf da #if DEMO_BUILD || DEVELOPMENT_BUILD || UNITY_EDITOR koşuluyla derlenir; üretim sürümünden dışlanır.

Genel akış üç katmanlıdır: **veri katmanı** (ScriptableObject varlıkları — RoomTemplateSO, SkillData, WeaponDatabaseSO) içeriği taşır; **runtime yönetici katmanı** (RoomRunDirector, sınıf controller'ları, Systems üzerindeki singleton'lar) bu veriyi okuyup davranışa çevirir; **sahne katmanı** (_Arena ve kalıcı Systems nesnesi) çalışan oyunu barındırır. Araç sınıfları (BuildModeController, DirectorMode, editör araçları) bu üç katmanın aynısını tüketir; bir tasarım değişikliği hem oyun hem araç tarafında anında yansır.

2.3 Sahne Akışı ve Manager Katmanı

Oyunun çalışma zamanı akışı birbirine bağlı birkaç sahne ve kalıcı bir yönetici katmanı üzerinden ilerler. Üst düzey akış şu biçimdedir:

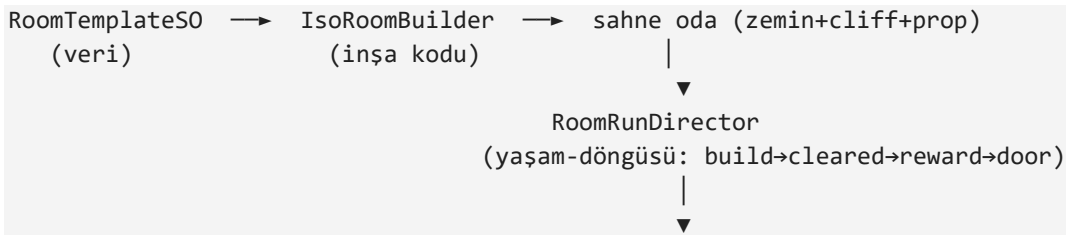


MainMenu sahnesi oyunun giriş noktasıdır; "Oyna" seçeneği oyuncuyu yürünebilir karakter seçim mekânı olan Attunement Chamber'a taşır. Asıl run, tek bir kalıcı arena sahnesi olan _Arena üzerinde gerçekleşir: her oda geçişinde önceki oda temizlenir ve bir sonraki oda verisinden yeniden kurulur. Bu "tek sahne, akan içerik" (same-scene streaming) yaklaşımı, oda başına sahne yükleme maliyetini ortadan kaldırır.

Kalıcı sistem nesneleri (Systems GameObject'i) sahne geçişlerinde yaşamaya devam eder; bu nesne üzerindeki singleton yöneticiler HUD, ses, meta-ilerleme ve oda akışı koordinasyonunu yürütür.

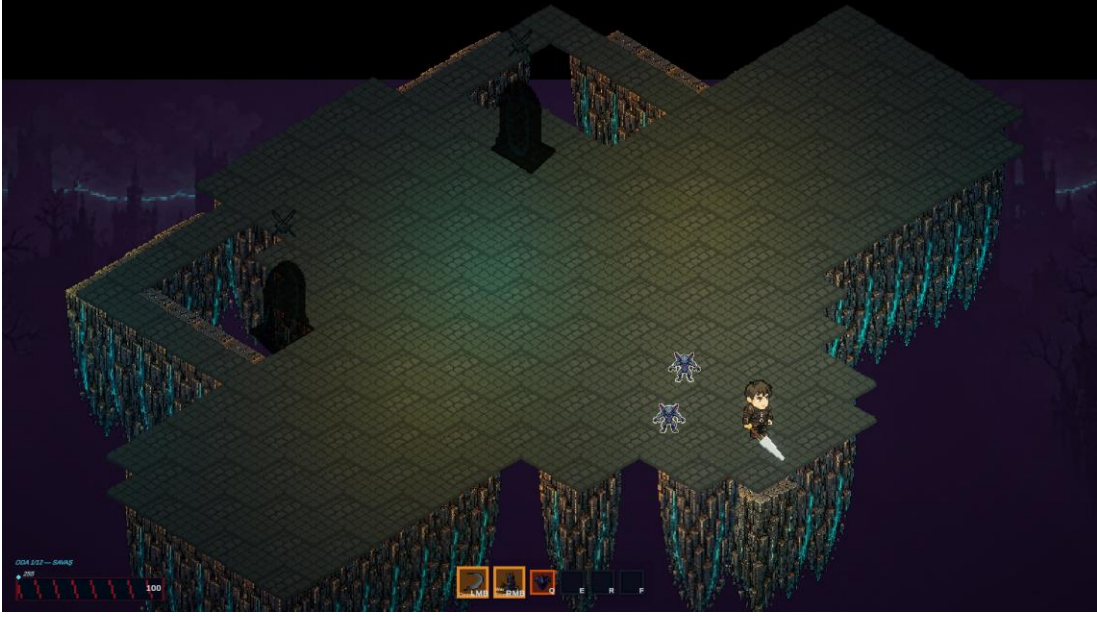
2.4 Veri → Sahne Boru Hattı

Mimarının omurgası, veriden sahneye uzanan deterministik bir boru hattıdır:



```
DungeonGraph.Generate(depthCount=5)  
(run-map: 8-11 düğüm, dallanan)
```

RoomTemplateSO, bir odanın salt-veri tanımıdır. Çalışma zamanında IsoRoomBuilder bu veriyi okur ve zemin karolarını, otomatik uçurum (cliff) sprite'larını ve bake edilmiş prop'ları sahneye yerleştirir. RoomRunDirector odanın yaşam döngüsünü (savaş → temizlik → ödül → portal) yönetir. Üst düzeyde DungeonGraph, run başında dallanan bir dungeon haritası üretir ve RoomRunDirector'a hangi odaların hangi sırayla kurulacağını bildirir. Bu üç katman birbirinden bağımsız değişebilir; bu da mimarinin hem okunabilirliğini hem genişletilebilirliğini artırır. Sistemin çalışır hâldeki görünümü Şekil 1'de verilmiştir.



Şekil 1: RIMA çalışan oyun ekranı — izometrik yüzen ada, HUD, oyuncu ve düşmanlar

3 VERİ MODELLERİ (ScriptableObject Tasarımı)

RIMA'da geleneksel bir ilişkisel veritabanı yoktur; içerik, Unity'nin ScriptableObject varlıkları olarak modellenir. Bu varlıklar projenin "veri katmanı"dır: kod-dışı, sürümlenebilir ve editör araçlarıyla düzenlenebilir. Her veri varlığı bir çalışma-zamanı sistemi tarafından okunur ve oyunun belirli bir parçasını besler; aşağıdaki tablo bu eşlemeyi özetler:

Veri Varlığı	Çalışma-Zamanı Sistemi	Kullanım
RoomTemplateSO	IsoRoomBuilder	Oda geometrisi, yürünebilirlik, kapı soketleri, bake'li prop'lar
PropDefinitionSO	BridsonPoissonAutoPlacer / Build Mode	Dekor görseli, collider ve kompozisyon-rolü; otomatik yerleşim + palet
SkillData / SkillDatabase	SkillController / DraftManager	Yetenek kimliği, tier, derinlik kilidi, sinerji pencereleri; draft havuzu
WeaponDatabaseSO / BasicAttackProfile	HandAnchorAttach / dövüş	Silah-el hizalaması, temel saldırı hasarı ve hit-pause tier'i

Örnek olarak, veri katmanının nasıl modellendiğini en temsil eden varlık bir oyun odasının tek-gerçek-kaynak (canonical) tanımı olan RoomTemplateSO'dur (JSON yalnızca dışa aktarım/araç formatıdır, ScriptableObject her zaman geçerli kaynaktır):

```
RoomTemplateSO
string   roomId           // benzersiz oda kimliği
RoomType roomType         // Combat / Elite / Reward / Boss / CharSelect
int      width, height    // ızgara boyutları
bool[]   walkableGrid     // hücre-başına yürünebilirlik
Vector2Int playerSpawn    // güney giriş noktası (player_spawn_01)
DoorSocket[] exitSockets  // door_NW_01 / door_N_01 / door_NE_01
PropPlacement[] bakedProps // önceden hesaplanmış prop konum+tip
DifficultyTag difficulty   // zorluk etiketi
```

Yetenek tarafında kritik bir tasarım ayrımı SkillDatabase.GetPool yönteminde gizlidir: veritabanındaki 111 skill kaydının yalnızca çalışır implementasyona sahip olanları (isImplemented) draft havuzuna girer; tasarım envanterindeki kalan kayıtlar editör ekranında görünür ama run içi seçime asla teklif edilmez. Bu veri modelleri sayesinde projenin tüm içeriği kod değiştirmeden düzenlenebilir; yeni bir oda, prop, yetenek veya silah eklemek bir veri varlığı oluşturmaktan ibarettir.

4 ANA SİSTEMLER VE ÖZELLİKLER

4.1 Tam Oynanabilir Döngü

RIMA, oynanabilir tam döngüye sahip çalışan bir prototiptir. Oyuncu Ana Menü'den başlar, yürünebilir Attunement Chamber'da bir sınıf seçer, oda oda ilerleyen bir run oynar, her üç odada bir 3-kart skill draft ekranıyla karşılaşır, dallanan Rift portallarıyla rota kararları verir, boss karşılaşmasına ulaşır ve run sonunda kazandığı Shattered Echo ile bir sonraki sınıfın kilidini açabilir. Bu döngünün tüm halkaları çalışır ve birbirine bağlıdır.

Attunement Chamber — Yürünebilir Karakter Seçimi

RIMA'nın karakter seçim ekranı statik bir menü değil, gerçek bir odadır. Oyuncu son oynadığı sınıfın bedeninde spawn olur ve WASD ile serbestçe dolaşır. Oda çevresindeki pedestal'larda sınıflar donmuş "echo"lar gibi bekler: açık sınıflar taş-grisi, kilitli olanlar opak siyah silüetlerdir. Bir silüete yaklaşıldığında sınıf paneli kayarak girer; oyuncu bir combat dummy üzerinde vuruş hissini deneyebilir. [G] tuşu karakteri o sınıfa dönüştürür; kuzey duvarındaki Rift kapısından çıkmak seçimi onaylar.

Editör Deneme Alanı (Chamber [K])

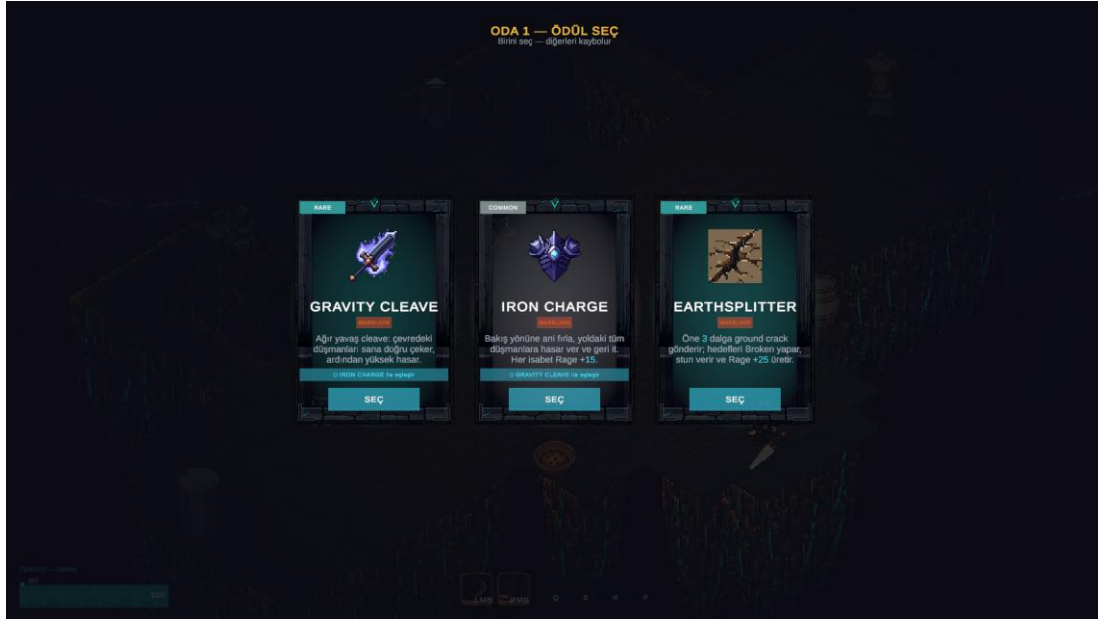
Attunement Chamber'da oyuncu [K] tuşuyla aktif sınıfın uygulanmış tüm yetenek listesine (SkillDatabase) erişir ve istediği yeteneği Q/E/R/F yuvalarından birine atayarak combat dummy üzerinde deneyebilir. Slot atamaları yalnızca oda-içi oturumda bellekte tutulur; run başladığında sıfırlanır ve koşu yükü ile draft akışına dokunmaz. Bu özellik, özellikle geniş yetenek havuzu olan sınıflarda oyuncunun run öncesinde sinerji olasılıklarını keşfetmesini sağlar.

Dövüş (Combat)

Oda aktif olduğunda düşman dalgaları sahneye girer. Oyuncu sınıfa özgü saldırı zinciri (LMB combo) ve yetenek seti (Q/E/R/F) ile dövüşür. Düşmanlar hasar aldıklarında geri savrulur; ağır saldırılar tam bir knockdown animasyonu başlatır — düşman yere düşer ve kalkış animasyonu bitene kadar i-frame (geçici dokunulmazlık) korumasına sahiptir. Son düşman da devrildiğinde oda "Cleared" durumuna geçer.

3-Kart Yetenek Seçimi (Skill Draft)

Her üç odada bir, oda ortasında bir ödül nesnesi belirir. [G] ile alındığında üç yetenek (skill) kartı açılır; her kart sınıfa ait ya da nötr bir pasif yetenek sunar ve araç ipucu (tooltip) mevcut yapıyla (build) sinerji noktalarını gösterir. Oyuncu birini seçer ve yetenek kalıcı olarak koşuya (run) eklenir. Bu akış Şekil 2'de gösterilmiştir.



Şekil 2: 3-kart skill draft ekranı ve ödül akışı

Dallanan Rift Portalları

Oda temizlenip ödül alındıktan sonra arka kenarda 1–3 arasında Rift portalı açılır (kod düzeyinde door_NW_01 / door_N_01 / door_NE_01 soketleri). Her portalın simgesi hedef oda türünü işaret eder: olağan savaş, daha zorlu ama daha iyi ödüllü elite, ya da doğrudan ödül odası. Oyuncu hangi portaldan geçeceğine karar vererek run'un kısa vadeli akışını ve sonundaki build'i biçimlendirir.

Boss Karşılaşması

Oturum (run) sonuna doğru, harita grafiğinde boss düğümüne ulaşıldığında boss arenasına giden portal açılır. Boss arenası boyut ve zorluk açısından standart odalardan ayrışır. Demo kapsamındaki boss, mevcut hollow_hulk sprite'nın büyütülmüş versiyonuyla temsil edilir ve telegraph sistemiyle altı saldırıya bağlanmıştır (bkz. §4.4). Demo karşılaşmasının kod-tarafı adı Penitent Sovereign'dır. Tasarım vizyonundaki nihai boss — "The Architect" — zengin bir faz yapısıyla yol haritasındadır.

Zafer / Ölüm ve Meta-İlerleme

Oturum sonunda oyuncuya kazandığı Shattered Echo miktarı gösterilir; hesap formülü $\text{odaSayısı} \times 3 + \text{öldürmeSayısı} / 5$ olup run başına 5–60 Echo ile sınırlıdır. Biriken Echo'lar bir sonraki oturumda kilitli sınıf silüetlerinden birini açmak için harcanır.

4.2 Build Sistemi: Skill Draft, Tier Kilitleri ve Sinerji

Skill'ler beş tier'a ayrılır ve draft sırasında görünme olasılıkları run derinliğine göre kilitlenir:

Tier	Görünme Ağırlığı	Run Derinliği Kilidi
Common (Yaygın)	55	Kilitsiz, ilk odadan itibaren
Rare (Nadir)	27	Kilitsiz, ilk odadan itibaren
Epic	12	Oda 3 ve sonrası
Mythic	5	Oda 3+, yalnızca birincil sınıf
Legendary	3	Oda 7 ve sonrası

Bu dağılım bilinçlidir: run'un ilk yarısında oyuncu temel kimliğini belirleyen yaygın/nadir skill'leri seçer; run ilerledikçe sinerji noktaları netleşir ve gerçek güç sıçramaları olan Epic+ tier'lar devreye girer. Legendary, build'in yönü belirginleştğinde son çeyrekte ortaya çıkar. Skill bar slotları da bir güç eğrisi olarak doldurulur: açılış draft'ı Q slotuna yerleşir; sonraki oda-temizleme draft'ları sırasıyla E, R ve F slotlarını doldurur; boss sonrasında iki yedek slot (Z, X) açılır.

Build sisteminin ikinci katmanı **chain window** (zincir penceresi) mekanizmasıdır: beş farklı zincir penceresi tanımlıdır. Belirli skill'ler ard arda tetiklendiğinde birbirinin etkisini çarpar; draft ekranında bir kart aktif bir zincir penceresiyle etkileşiyorsa bu sinerji küçük bir görsel chip olarak gösterilir.

4.3 Sınıf Tasarımı: Warblade Örneği

Veri modeli on sınıfı destekler; demo'da Warblade ve Elementalist varsayılan açık ve uçtan uca oynanabilir, Ranger ile Shadowblade aynı pipeline üzerinden etkinleştirilebilir durumdadır. Her sınıf farklı bir kaynak mekanizmasına ve oynanış diline sahiptir.

Warblade, "yaklaş, sabitle, zırh kır, infaz et" mantığıyla çalışan ağır bir savaşçıdır. Kaynağı, yalnızca hasar vererek ve kitle kontrolü uygulayarak dolan, savaş dışında pasif eriyen bir Öfke göstergesidir (Rage bar); bu, oyuncuyu sürekli temasa zorlar. Yetenek seti ("Iron Charge", "Gravity Cleave", "Iron Counter") savaşı düşmanı Broken veya Sundered durumuna yönlendirir; bu durumlardaki bir hedefe imza yetenek "Death Blow" uygulanabilir — Öfke göstergesini boşaltarak %400+ hasar veren bir infaz darbesi. Öfke göstergesi dolduğunda [V] ile "Bladestorm" ultimate devreye girer. Warblade sınıfının temel sprite'ı Şekil 3'te görülmektedir.



Şekil 3: Warblade sınıfı — omuz zırhlı silahsız temel sprite; silah mount sistemi Şekil 8'de

4.4 Düşman AI, Dalga Sistemi ve Boss Telegraph

Düşman dalgaları EncounterController tarafından bağımsız yönetilir; tüm dalgalar bittiğinde RoomRunDirector'a "temizlendi" sinyali gönderilir. Bu ayrışma, düşman dalga tasarımının oda mimarisinden bağımsız güncellenmesini sağlar.

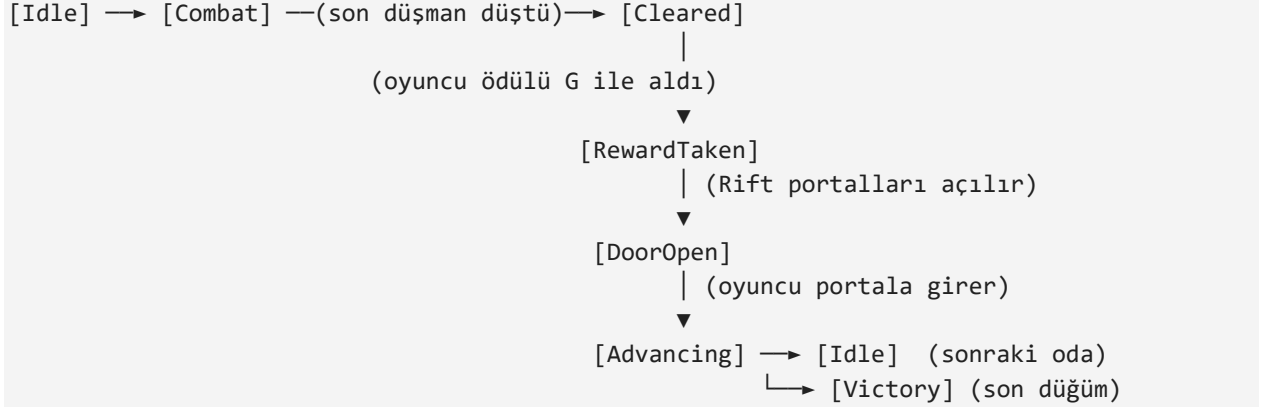
Düşman AI'sinin temelini BaseMobBehavior oluşturur. Algılama menzili (detectionRange) ile üretim (spawn) mesafesinin uyumu, düşmanların oyuncuyu doğru biçimde fark edip kovalaması için kritik bir parametredir; bu uyum veri-güdümlü doğrulamayla ayarlanmıştır.

Boss telegraph sistemi, ARPG konvansiyonuna uygun olarak saldırı öncesi tehlike alanlarını görselleştirir. Mevcut motor yeniden kullanılarak altı boss saldırısına bağlanmıştır: HolyLash (koni), FractureStrike (daire), ChainExplosion (gecikmeli halka), SovereignsWrath (dış halka + yeşil güvenli-bölge), FractureCharge (atılım çizgisi) ve ShackleThrow. Windup zamanlaması saldırı anıyla bit-bit senkronizedir;

telegraph-hasar uyumsuzluğu (görselden farklı zamanda vuran saldırı) önlenmiştir. SovereignsWrath'ın yeşil güvenli-halkası, tehlike halkasından ayrı renkte çizilerek oyuncuya "nereye durmalı" bilgisini okunur biçimde verir.

4.5 Oda Akışı State Machine

Bir RIMA odasının yaşam döngüsü 7 durumlu bir durum makinesiyle yönetilir:



RoomRunDirector bu yedi durumu yönetir ve geçişleri kasıtlı olarak tek yönlü tutar: savaş bitmeden ödül alınamaz, ödül alınmadan portal açılmaz, portala girilmeden sonraki oda kurulmaz. Bu tasarım hem net bir ilerleme hissi sağlar hem de yarış koşullarını (race condition) önler. Ölüm, Victory dışındaki herhangi bir durumda gerçekleşebilir ve run'u Death Screen'e taşır.

Hareket güvenliği — yetenek tabanlı yer değiştirme. Blink (ışınlanma), Iron Charge ve Blade Rush gibi yer değiştirme yetenekleri, standart oyuncu hareketiyle aynı WalkabilityMap.IsDashableWorld / ClampVelocityToWalkable boru hattından geçer; böylece tüm yetenek tabanlı yer değiştirmeler oda sınırlarına saygı duyar ve oyuncuyu harita dışı geçersiz alana (void) göndermez.

4.6 Oyun Hissi (Game-Feel) Katmanı

Oyun hissi (game-feel) katmanı, bir aksiyon oyununun mekanik doğruluğunun ötesinde inandırıcı hissetmesini sağlar. RIMA'da bu katman dört oyuncu-deneyimi alanından oluşur; her biri minimum yeni kaynakla maksimum geri bildirim sağlamayı hedefler:

- **Vuruş hissi.** Hafif/ağır/infaz vuruşları için kademeli zaman dondurması (0.03 / 0.06 / 0.10 s) ve eşleşen ekran sarsıntısı; hasar sayıları, hit-stop ve kamera-punch'tan oluşan CombatJuice katmanı. Tüm efektler tek bir FeelToggle bayrağıyla erişilebilirlik için kapatılabilir.
- **Görsel netlik.** EnemyReadable.shader ile her renkte gerçek silüet outline ve yükseltilmiş ortam ışığı düşmanları öne çıkarır; URP Volume son-işleme (Bloom, Color Adjustments, Tonemapping, Vignette) ve 2D ışık "yüzen ada + void" kanonunu koruyarak sahneyi okunur kılar. Broken/Sundered düşmanların üzerinde, cyan'dan ayrı tutulmuş altın renkli "[RMB] İnfaz" dünya-prompt'u belirir.
- **Ses.** İki katmanlı mimari: 18 CCO klip dokuz bağımsız SFX kanalına (swing, hit, death, execute, oda-clear, draft, ambient, knockdown, UI) ve sahne-geçişinde ambient sızıntısını önleyen bir yaşam-döngüsü korumasına bağlıdır; altta düşük volume CCO dungeon-ambience müzik yatağı döngüdedir. Özgün müzik ve sınıfa-özgü SFX gelecek kapsamındadır.
- **Skill sunumu ve arayüz.** Karakter animasyonu yerine VFX + kod-tween kombinasyonu kullanılır: her skill bir çıkış hareketi, mevcut VFX envanterinden bir efekt (slash_arc, glacial_spike, meteor, 8-

yön fireball) ve hit-pause + sarsıntı + HitFlash impact reçetesiyle tanımlanır; böylece sıfır yeni sprite ile skill eylemleri okunur kılınır. Skill çubuğu sürükle-bırak ile kalıcı yeniden sıralamayı ve tüm slotlar için sayısal bekleme-süresi göstergesini destekler.

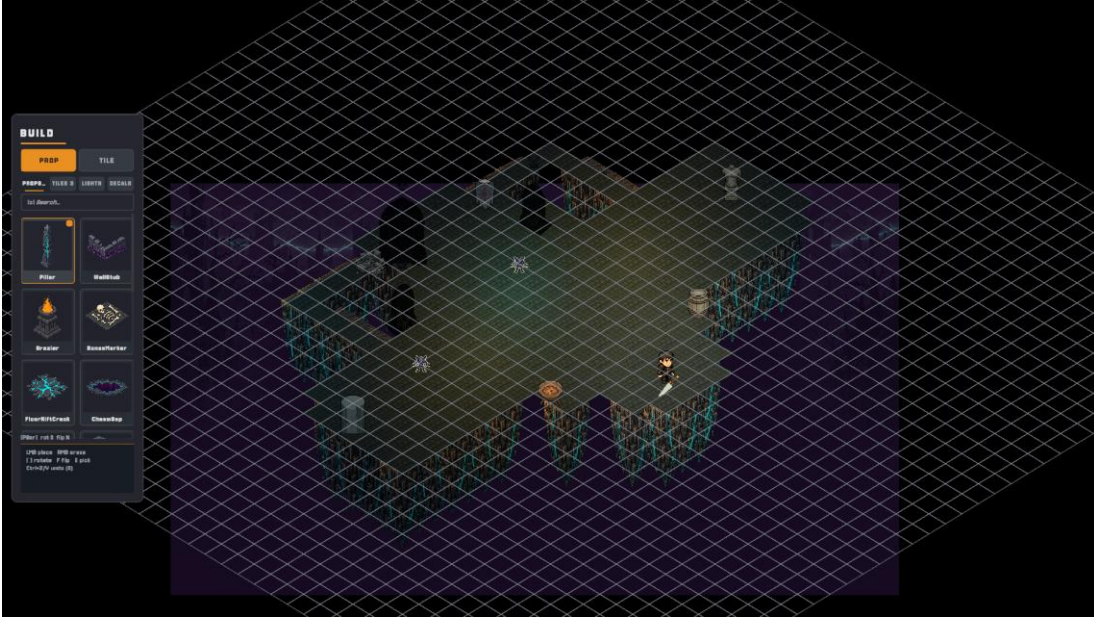
5 OYUN-İÇİ GELİŞTİRME ARAÇLARI

Projenin merkezî mühendislik katkısı, oyun çalışırken — Unity editörünü açıp kapatmadan — tasarım iterasyonu yapılmasını sağlayan oyun-îçi araç katmanıdır. Bu araçlar graphify kod-graflı analizinde en bağılı (god-node) sınıflar arasındadır ve "bu proje bir geliştirme ortamıdır" tezinin somut kanıtıdır.

5.1 Build Mode (F2) — Edit-to-Play

Build Mode, oyun çalışırken F2 tuşuyla açılan oyun-îçi seviye editörüdür. Açıldığında oyun duraklatılır; geliştirici nesneleri (prop) yerleştirir, zemini düzenler, odayı kurar — sonra modu kapatıp **aynı odayı anında oynar**. Bu "düzenle-ve-oyna" (edit-to-play) döngüsü, geleneksel "Unity'de düzenle → kaydet → Play'e geç" döngüsünü ortadan kaldırır.

BuildModeController (graphify'da en bağılı araç düğümlerinden), tek-sahip bir F2 kayıt defteri üzerinden çalışan geçiş mantığını, üzerinde çalışılan WorkingTemplate'i ve düzenle↔oyna geçişini yönetir. Prop yerleştirme BuildPlacementController tarafından bir palet → hayalet (ghost) → tıkla akışıyla gerçekleştirilir. Palet veri-güdümlüdür: BuildModeAssetCatalog, PropRegistrySO'dan beslenir; palet 9'dan 19 prop'a çıkarılmıştır. Bu araç, Bölüm 2'de tanımlanan veri-güdümlü mimarinin doğrudan bir tüketicisidir: aynı RoomTemplateSO verisini hem oyun hem editör paylaşır, dolayısıyla değişiklikler anında yansır. Şekil 4, Build Mode arayüzünü göstermektedir.



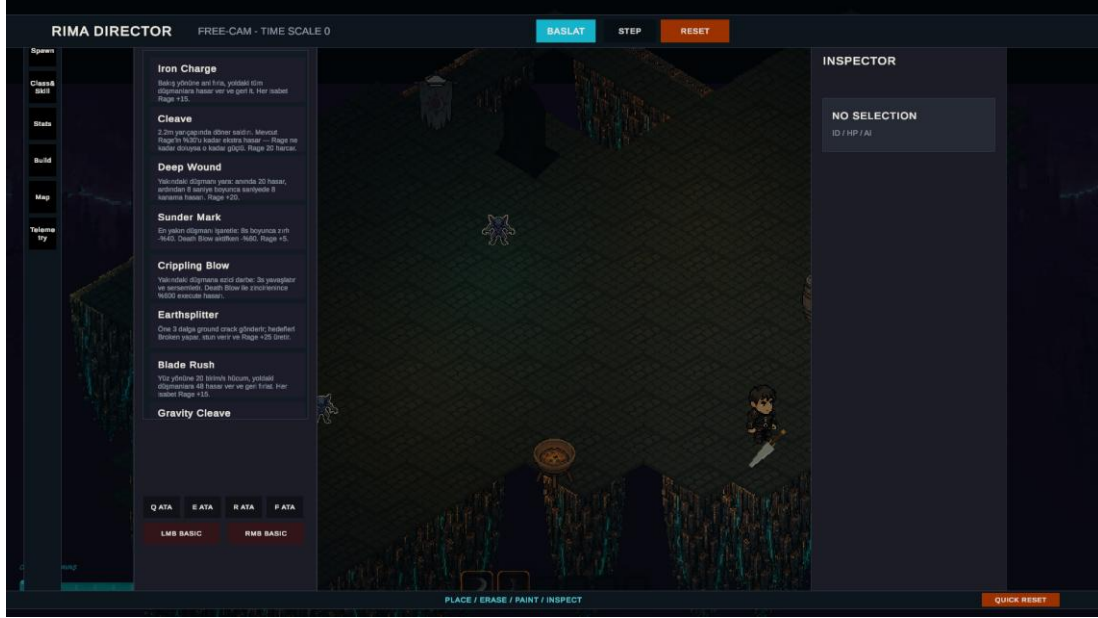
Şekil 4: Build Mode (F2) — oyun-îçi seviye editörü, prop paleti ve ızgara overlay'i

5.2 Director Mode — Runtime UI Factory

Director Mode (DirectorMode.cs), sahne kurulumu gerektirmeden çalışma zamanında bir arayüz üreten geliştirici konsoludur. Açıldığında sekmeli bir IDE-dock düzeninde çalışır ve şu panelleri sunar: Stat slider'ları (oyuncu/düşman istatistiklerini canlı ayarlama), Spawn (düşman üretimi), Telemetry (CSV çıktısı), Prop, Map ve Free-cam. Bu kapsamda Director Mode, eski hata-ayıklama katmanı görünümünden profesyonel bir dock düzenine (görüntü alanı ~%57, SDF yazı tipi, ScrollRect'li

kaydırılabilir yetenek listesi) taşınmıştır. Director Mode, kod-grafında en yüksek bağlantı derecesine sahip düğümdür (bkz. Bölüm 10), çünkü oyunun hemen her sistemine canlı bir gözlem ve müdahale arayüzü sağlar. Şekil 5, Director Mode panellerini göstermektedir.

Araç sağlamlığı kapsamında çift-yuva reddi de eklendi: zaten bir yuvada bulunan yetenek bileşeni ikinci bir yuvaya atanmak istendiğinde, atama bileşeni oluşturulmadan önce reddedilerek mevcut hata-bildirim arayüzü üzerinden kullanıcı uyarılır; bu sayede aynı bileşenin iki yuva tarafından paylaşılıp bekleme sürelerini bozduğu öksüz bileşen sızıntısı önlendi.



Şekil 5: Director Mode — runtime UI dock (Stat / Spawn / Telemetry panelleri)

5.3 F1 Debug Paneli

F1 tuşuyla açılan hafif bir geliştirme paneli; God Mode, Kill All ve oda-atlama (room-skip) gibi hızlı geliştirme komutları sunar. Build Mode ve Director Mode'dan farklı olarak F1, hızlı tek-seferlik müdahaleler için tasarlanmıştır.

Bu üç araç birlikte, "editörsüz geliştirme" iş akışını oluşturur: geliştirici çoğu tasarım iterasyonunu oyun çalışırken yapar, bu da test-değiştir-test döngüsünü belirgin biçimde kısaltır.

6 VERİ-GÜDÜMLÜ ODA SİSTEMİ VE ODA İNŞA HATTI

6.1 Veriden Odaya: Sahneden Veriye ve Otomatik Uçurum Yerleşimi

Geleneksel yaklaşımda her oda ayrı bir sahne dosyasıdır; 30 oda için 30 sahne ve her güncelleme için 30 açma-kapama döngüsü demektir; tek geliştirici için bu yük orantısızdır. RIMA'da her oda bir RoomTemplateSO veri yapısında tanımlanır; görsel yapı (zemin, kenar, uçurum, prop'lar) oyun başlayınca bu veriden çalışma zamanında inşa edilir. Bunu mümkün kılan IsoRoomBuilder, tek bir _Arena sahnesi üzerinde çalışır; her geçişte önceki oda temizlenir, yenisi sıfırdan kurulur.

RIMA'nın görsel dili "yüzen izometrik ada" üzerine kuruludur: her oda, boşlukta asılı koyu granit bir platformdur. Bu görünümün en zorlu katmanı tamamen otomatik olan uçurum (cliff) yerleşimidir.

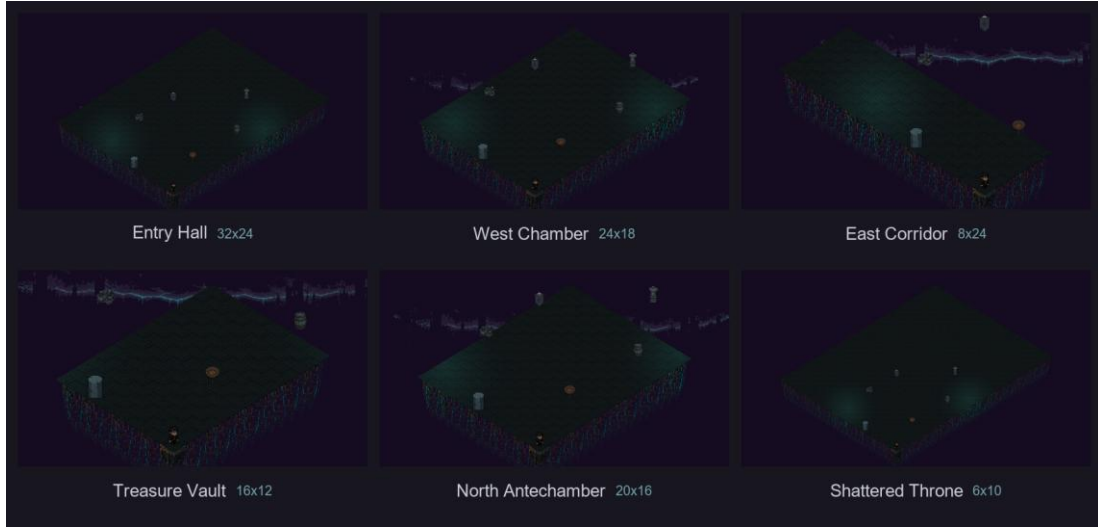
Çalışma zamanında IsoRoomBuilder, zemin ızgarasını tarar ve her hücrenin SW (güneybatı) ile SE (güneydoğu) komşularının void olup olmadığını kontrol eder; güney cephesindeki hücrelere "içeri kıvrılma" (inward tuck) vektörüyle ön-yüz uçurum sprite'ı yerleştirir. Yön-bazlı komşu kontrolünün özü şu mantıktadır:

```
foreach (cell in floorCells):
    bool voidSW = IsVoid(cell + SW)
    bool voidSE = IsVoid(cell + SE)
    if (voidSW || voidSE):
        // güney cephesi → derinlik koruyan inward-tuck
        PlaceCliff(cell, tuck: true)
    else if (HasAnyVoidNeighbor(cell)):
        // diğer yönler → standart kenar uçurumu
        PlaceCliff(cell, tuck: false)
```

Bu yön-ayrımı kritiktir: yön bilgisi olmadan her void-komşu hücreye aynı uçurum sprite'ı yerleştirilseydi güney cephesinde zemine taşan yığılmalar oluşurdu. Editör tarafında CliffAutoPlacer sekiz izometrik yön için daha kapsamlı bir directional mantık sunar ve şablonları editör zamanında hazırlar. Her iki sistemde de geliştirici tek bir uçurum karosunu elle yerleştirmez.

Adanın fiziksel sınırı üç boyutta zorlanır: **oda geometrisi** (statik sınır doğruluğu — donut odaların iç deliği için collider üretimi), **hareket güvenliği** (knockback itmesi, knockdown yayı ve elite-teleport hedeflerinin yürünebilir hücrelere clamp'lenmesi, diyagonal köşe-kesmenin engellenmesi) ve **doğrulama** (tünelleme analizi: kare başına kat edilen mesafe hücre boyutunun çok altındadır). Walkable test grupları tamamen yeşildir.

Şekil 6, Act-1'in altı odasının IsoRoomBuilder ile çalışma zamanında inşa edilmiş oyun-içi görünümünü bir arada gösterir; BuildCliffs'in salt-veri taban hücrelerinden ürettiği prosedürel uçurum kenarı, harita dışı geçersiz alan (void) arka planı ve 2B ışık birleşince ada-görünümünün tümüyle veri + inşa düzeneğinden geldiği — elle yerleştirilmiş tek bir karo olmadan — doğrudan gözlemlenebilir.



Şekil 6: Act-1'in 6 odası IsoRoomBuilder ile üretilen cliff-island oyun-içi görünümü — procedural uçurum kenarı + void arka plan + 2D ışık

6.2 Otomatik Prop Yerleşimi

Her şablon bir prop listesi taşır. Konumlar elle değil, BridsonPoissonAutoPlacer editör aracıyla (#if UNITY_EDITOR) algoritmik üretilir ve RoomTemplateSO'ya bake edilir; çalışma zamanında IsoRoomBuilder bu önceden hesaplanmış konumları okur. Bridson-Poisson algoritması, yürünebilir hücreler üzerinde minimum örnekleme mesafesi r garantisiyle noktalar üretir; böylece prop'lar üst üste yığılmaz, organik dağılır. Bir kompozisyon-rol haritası ek kısıt getirir: oda merkezi "temiz merkez" olarak düşük yoğunlukta tutulur (dövüş alanı), kapı girişleri sıfır yoğunlukla korunur, kenar kuşakları dekorasyona ayrılır. Combat ve Elite oda tiplerinde prop'lar çalışma zamanında devre dışı bırakılarak savaş okunabilirliği artırılır.

6.3 Dış Kaynaklı İçerik ve JSON İçe Aktarma

26 oda şablonunun yaklaşık 11'i dahili tasarımıdır; 15'i prosedürel JSON oda şablonu olarak harici araçlarla üretilip içe aktarılmıştır. Süreç gözetimsiz bırakılmamıştır: geçerli bir tasarım, ASCII ızgara düzeni, kapı soketleri, zorluk etiketi ve prop listesi içeren belirli bir JSON şemasına uymalıdır; bu format doğrudan RoomJsonImporter aracına beslenebilir. Üretilen tasarımlar bir eleme sürecinden (çeşitlilik, zorluk dengesi, tematik uygunluk) geçirilmiş, seçilenler mevcut odalarla birleştirilerek 26 şablonluk havuz oluşturulmuştur.

Oda Havuzu Özeti	Değer
Toplam oda şablonu	26
Dahili tasarım	~11
Harici araçla üretilip içe aktarılan	15
QC uygulanan şablon	26

Bu odalar, harici araçlarla üretilen JSON şablonlardan RoomJsonImporter aracılığıyla içe aktarılır. Her oda dosyası zorunlu alanlar olan oda kimliği (room_id), ızgara boyutları (width/height), zemin malzeme bölgeleri (floor.zones) ve duvar prefab konumlarını içerir. Şema-doğrulamasından geçmeyen dosyalar içe aktarım aşamasında reddedilir.

JSON Oda Şeması — Temel Alanlar	Açıklama
schema_version	Format sürümü (doğrulama için)
room_id	Benzersiz oda tanımlayıcısı
room_type	spawn / combat / elite / reward / boss
width,height	Izgara boyutları (hücre)
floor.zones[]	Malzeme bölgeleri: rect + blend_edges
walls[]	Prefab konumları: prefab adı + (x, y) + rotation

Tekil oda dosyalarının yanında bir `_manifest.json`, odaların birbirine nasıl bağlandığını tanımlayan bir kapı grafiği (`door_graph`) taşır; bu graf doğrudan oturma-haritasının (`run-map`) dallanmasını besler. Giriş odası (`entry_hall`), Kuzey/Doğu/Batı dallarıyla altı-odalı bir bağlılık grafi oluşturur: `entry_hall` → `north_antechamber` → `shattered_throne` (dikey omurga); `east_corridor` → `treasure_vault` (doğu kolu); `west_chamber` → `north_antechamber` (döngüsel bağ).

6.4 Authored Portal Soketi Sistemi

Her `RoomTemplateSO`, oyuncunun giriş noktasını (`player_spawn_01`, yürünebilirlik doğrulamalı) ve arka kenarda üç çıkış soketi (`door_NW_01`, `door_N_01`, `door_NE_01`) tanımlar. Bu soketler validator kurallarına tabidir: her soket hücresi yürünebilir ve kuzey komşusu yürünemeyen kenar olmalı; güney koridoru (2 hücre) yürünebilir kalmalı; soketler birbirinden en az 3 hücre ayrı olmalıdır.

Portal açısı ile aktif çıkış slotu sayısı bağımsız kavramlardır. Görsel maliyeti sınırlamak için iki authored portal açısı kullanılır: N soketi için frontal kemer, NW için açılı kemer; NE, açılı kemerin çalışma zamanında yatay aynalanmasıyla (`flipX`) elde edilir. Çalışma zamanında `BuildExitDoors`, `dungeon graph` düğümünün çocuk dal sayısına bakar: tek çıkış için yalnızca N, iki çıkış için NW+NE (merkez boş, simetri korunur), üç çıkış için her üçü. Hiçbir şablon geçerli slot sayısını karşılayamazsa sistem sessizce başarısız olmaz: uyarı yazar, editör validasyonu düzeltmeyi zorlar ve merkez-anchor geri dönüş düzeneği fail-safe olarak devreye girer.

6.5 Geliştirme Araç Zinciri (Editör)

Oyun sistemleriyle paralel olarak bir editör araç zinciri tasarlanmıştır:

- **Map Designer:** Birleşik 8-sekmeli pencere (Rooms, Library, Floor, Cliff, Object, Portal, Light, Layers). Rooms sekmesi 6-modlu bir toolbar (Paint Walkable/Void, Set Entry/NW/N/NE) ve UI↔JSON çift-yönlü senkronizasyon içerir; her boya darbesi Unity Undo'ya entegredir. ScriptableObject canonical kaynaktır; JSON ~1 s debounce ile dışa aktarılır, diff-kontrolüyle gereksiz yazma önlenir.
- **Room Browser:** Tüm `RoomTemplateSO` dosyalarını listeler ve tek tıkla `_Arena`'da kurar; 26 şablonun tamamı bu araçla görsel doğrulanmıştır.
- **RoomJsonImporter:** JSON oda verisini şema-doğrulamasıyla `RoomTemplateSO`'ya dönüştürür.
- **RuntimeAssetRegistryBaker:** Zemin/uçurum/prop varlıklarının GUID, ad, etiket, katman ve tür meta verisini çalışma zamanına aktarır.
- **ScreenshotMode:** F12 ile açılan, 6 kamera preset'li, HUD'lu/HUD'suz, deterministik prop-seed'li sunum modu; aynı şablon + aynı seed her seferinde piksel-piksel aynı kompozisyonu üretir (imza-eşitlik testiyle doğrulanmıştır).

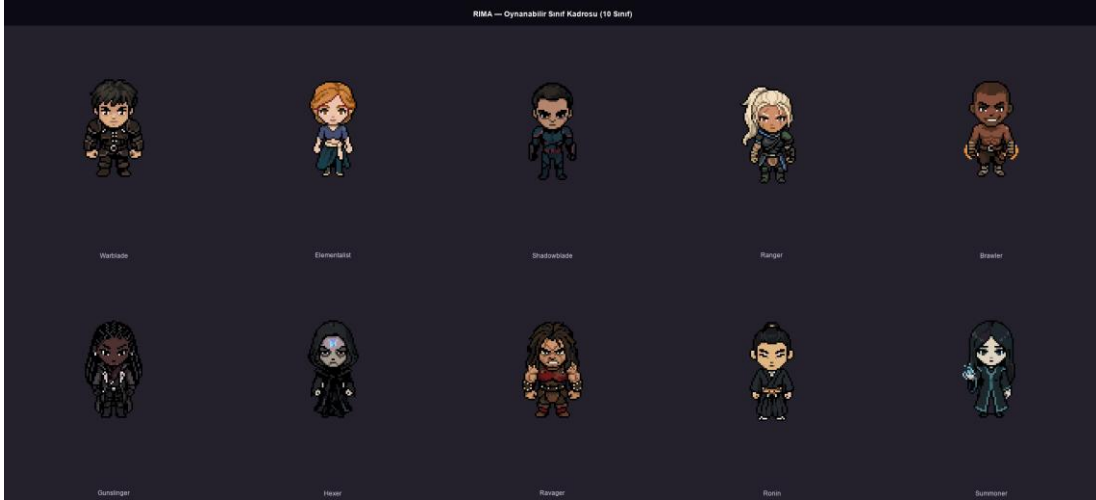
7 GÖRSEL ÜRETİM HATTI

7.1 Sanat Yönü

Görsel tasarım, izometrik yüzen adalar ve onları çevreleyen mor void teması üzerine kuruludur. Zemin koyu slate/granit tonlarındadır; koyu zemin karakter ve düşmanları zemine yapıştıran bir silüet okunabilirliği sağlar. Cyan (#00FFCC) enerji çatlakları ve rün parıltıları zinde odaları, karar noktalarını ve aktif objeleri işaret eder; ancak ekran alanının yalnızca %5–8'ine hâkim olacak biçimde kullanılarak atmosferin baskın rengi korunur. Kahverengi tonlar bilinçli olarak dışlanmıştır. Bu dil, proje başında belgelenmiş bir "sanat yönü kilidi" olarak sabitlemiştir.

7.2 Karakter Sprite Üretimi: 10 Sınıf × 8 Yön

Her sınıf sekiz yönde (K, KD, D, GD, G, GB, B, KB) idle sprite'a ihtiyaç duyar; toplam 80 sprite elle çizilemeyecek bir yükür. Çözüm PixelLab AI'dir. Sekiz yönün tamamı bağımsız üretilmez: K, KD, D, GD, G beş yön yapay zekâ ile üretilir; GB, B, KB ise yatay aynalama (flipX) ile türetilir. Bu, içerik maliyetini yaklaşık %37 düşürür ve aynalanan yönlerde stil tutarlılığını doğal olarak korur. Tuval boyutları sınıf silüetine göre değişir (120×120, 124×124 veya 128×128); görünür gövde her sınıfta ~64 piksel uzunluğundadır, kalan boşluk animasyon eylemleri için tampondur. 10 sınıfın güney-yön sprite'larının yan yana dizilimi Şekil 7'de görülmektedir.



Şekil 7: Oynanabilir sınıf dizilimi — 10 sınıfın idle_south sprite'ları

7.3 Piksel-Art İçe Aktarma Disiplini

Grafik kalitesini korumak için katı bir içe aktarma sözleşmesi uygulanır:

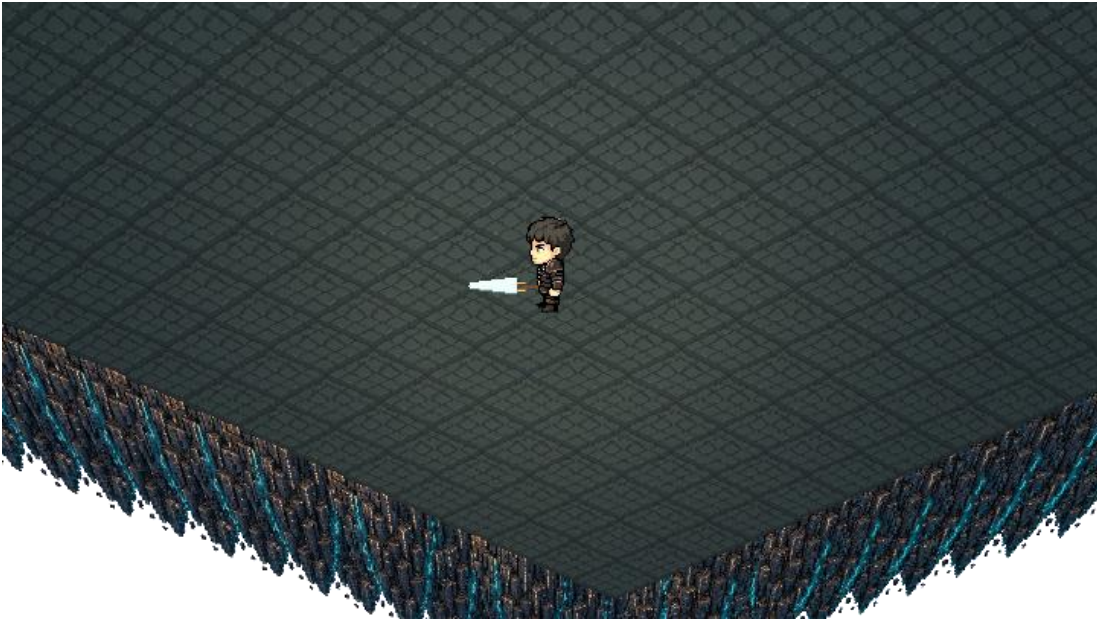
- **Filtre modu:** Point (Nearest Neighbor) — enterpolasyonu engeller.
- **PPU (piksel başına birim):** 64 — tüm sprite'larda ortak, ölçekleme buglarını önler.
- **Sıkıştırma:** Yok — Unity'nin lossy sıkıştırması piksel kenarlarını bozar.
- **Ölçekleme:** Tam sayı çarpanı — kesirli ölçekleme bulanıklaştırır.

Bu sözleşme yalnızca kural değil, fiilen hata yakalayan bir mekanizmadır: bir skill ikonu partisi Bilinear filtre ile gelip gözle görülür bulanıklığa yol açtığında bir QC geçişinde tespit edilmiş ve etkilenen grup

toplu olarak Point'e çevrilmiştir. Ayrıca her karakter sprite'ının pivotu ayak tabanına ayarlanır; proje başında 82 rotasyon sprite'ı varsayılan 0.5 (merkez) pivotla geldiğinden karakterler "havada asılı" görünüyordu. Her sprite'ın alpha kanalı taranarak ayak hizası ölçülmüş ve pivot oraya taşınmıştır. Oyuncu ve 16 mob sprite'ı tek bir "Entities" sorting layer'ına alınmış; IsoSorter Y-eksenine dayalı derinlik sıralamasını çalışma zamanında yönetir.

7.4 Silah Mount Sistemi

Karakter ve silah sprite'ları ayrı üretilir ve çalışma zamanında birleştirilir. HandAnchorAttach, WeaponDatabaseSO'daki anchorOffset ve gripOffset verisini kullanarak silahı karaktere prosedürel olarak bağlar; ön/arka el çevirme (flip) mantığı yön değişiminde silahın doğru tarafta görünmesini sağlar. Yön başına el-yuvası hizalaması demo kapsamı dışında bırakılmış; tam hizalama sonraki geliştirme kapsamı olarak planlanmaktadır. Bu birleştirme sistemi Şekil 8'de gösterilmektedir.



Şekil 8: Silah mount — prosedürel silah bağlama sistemi; karakter ve silah sprite'ları çalışma zamanında `HandAnchorAttach` ile birleştirilir

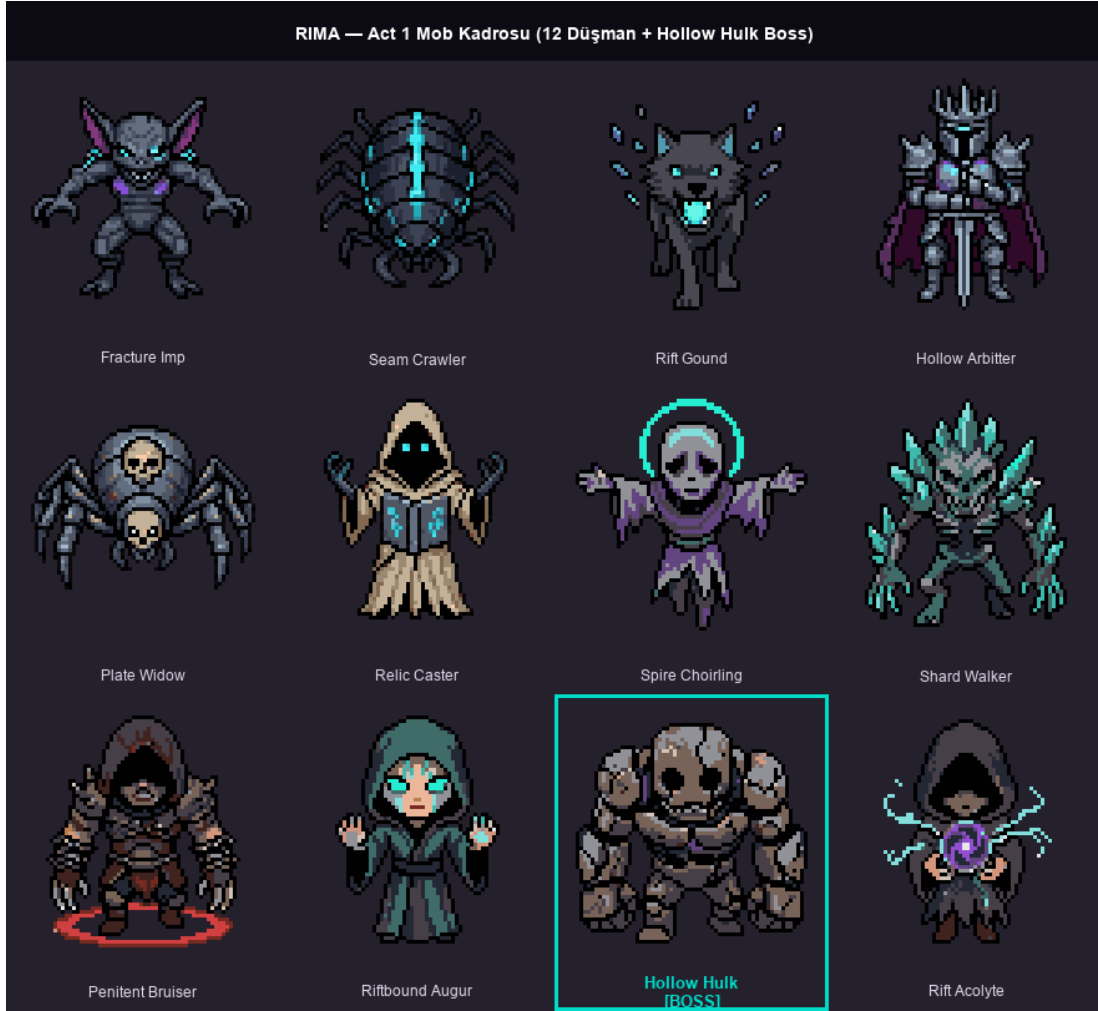
7.5 Ortam Asset'leri ve İki Araç Ayrımı

Karakter üretimi yalnızca PixelLab, ortam/UI üretimi yalnızca Imagen (agy boru hattı) ile yapılır; bu ayrım teknik değil estetikdir. Farklı motorlardan gelen sprite'lar stil uyumsuzluğuna yol açabileceğinden, oyuncunun sürekli baktığı karakterler tek kaynakta tutulur. Toplamda 115 PNG staging alanında birikmiş; bir bölümü doğrudan asset, bir bölümü stil/palet referansı olarak kullanılmıştır. Her iki araçtan gelen varlık, kullanılmadan önce görsel QC'den (sınır keskinliği, arka plan şeffaflığı, palet tutarlılığı, orantı) geçirilir.

7.6 Demo Düşman Kadrosu

Demo'da 12 düşman sprite'ı kullanılır; tümü ortak içe aktarma sözleşmesini paylaşır (64×64, PPU 64, Point, sıkıştırma kapalı), chibi high-top-down (~65°) stil, 1 piksel dış çizgi (outline), soğuk slate-gri + siyan-Rift paleti. Kadro: fracture_imp, relic_caster, seam_crawler, plate_widow, hollow_arbiter, rift_gound,

spire_choirling, shard_walker, penitent_bruiser, riftbound_augur, hollow_hulk (demo boss'u olarak da büyütölmüş kullanılır) ve rift_acolyte. Düşman kadrosunun kontak sayfası Şekil 9'da görölmektedir.



Şekil 9: Act-1 düşman kadrosu kontak sheet'i — 12 sprite, Hollow Hulk boss olarak işaretli

8 YAPAY ZEKÂ DESTEKLİ ÇOK-AJANLI GELİŞTİRME METODOLOJİSİ

8.1 Gerekçe: Kaynak Kısıtları

Bölüm 1'deki kaynak kısıtı — tek geliştiricinin programcı, tasarımcı, sanatçı ve QA rollerinin tamamını üstlenmesi — bu metodolojinin gerekçesidir. Süreç, bir dil modeline komut verip çıktısını doğrudan kabul etmekten farklıdır: farklı rollere sahip ajanların belirli kurallara göre etkileştiği, her kararın belgelendiği ve her çıktının bağımsız doğrulandığı bir yazılım mühendisliği süreci tasarlanmış ve işletilmiştir. Ajanlar araçtır; süreci tasarlayan ve yöneten geliştiricidir. Yaklaşım, sektörde hızla olgunlaşan agentic-AI pratiklerini akademik bir prototip üretimine uyarlama denemesidir.

8.2 Sanal Ekip Yapısı

Altı temel rol tanımlanmış ve farklı araçlarla doldurulmuştur:

Rol	Kategori	Ne Yapar
Orkestratör	Büyük dil modeli (koordinasyon)	Görev dağıtımı, sıralama, sentez, nihai karar
Kod Üretim Modülü	Büyük dil modeli (uygulayıcı)	Kod yazma, Unity değişiklikleri, mekanik toplu iş
Mimari Çapraz-Doğrulama	Büyük dil modeli (danışman)	Tasarım kararları, mimari değerlendirme, risk analizi
Kalite Kontrol Ajanı	Büyük dil modeli (denetçi)	Çıktı kalite kontrolü, hata yakalama (yazar ≠ denetçi)
Bilgi Tabanı	NotebookLM (MCP üzerinden)	Tasarım kararlarının kalıcı, sorgulanabilir arşivi
Ortam Sürücüsü	Unity MCP, PixelLab MCP	Ajanların Unity editörünü ve görsel hatları doğrudan sürmesi

Sürecin temeli yazar-denetçi ayrılığıdır: **bir ajandan çıkan iş, o ajanın kendisi tarafından onaylanamaz.** Bilgi tabanı rolündeki NotebookLM, projenin belleği olarak işlev görmüş; her ajan büyük bağlam okumak yerine bu tabanı MCP üzerinden sorgulamıştır; bu hem token tüketimini azaltmış hem de kararların oturumlar arası tutarlılığını sağlamıştır. MCP ayrıca ajanların Unity Editor üzerinde doğrudan işlem yapmasını (sahne nesnesi oluşturma, SO güncelleme, play-mode kontrolü) ve PixelLab'de sprite işi tetiklemesini mümkün kılmıştır.

8.3 Süreç Kuralları

Sürecin en özgün yönü, ajanların yeteneklerinden çok onları yönetmek için geliştirilen kurallardır; bu kurallar denemeler ve gözlenen hatalar üzerinden belgelenip zorunlu kılınmıştır:

- Görev dosyaları:** Her iş bir görev belgesiyle başlar (ne yapılacak, hangi dosyalara dokunulacak, neye dokunulmayacak, başarı kriteri). Ajan ya kriteri karşılar ya "BLOCKED" yazar; sessizce yarım iş kabul edilmez.
- Karar dökümanları:** Büyük her mimari/tasarım kararı STAGING/ altına alternatifleri ve gerekçeleriyle belgelenir; 100'ü aşkın karar belgesi yazılmıştır.
- Cross-review zorunluluğu:** Yazar ve reviewer aynı ajan olamaz. Örneğin Kod Üretim Modülü'nün yazdığı knockdown sistemi Mimari Çapraz-Doğrulama Ajanı tarafından, Mimari Çapraz-Doğrulama kanalıyla üretilen bir bileşen ise Kod Üretim Modülü tarafından incelenmiştir.

- **Doğrulama kanıtı:** Bir iş "bitti" sayılmadan önce derleme hatası olmadığı, ilgili testlerin geçtiği ve oyun içinde doğrulandığı raporlanmalıdır.
- **Unity'ye tek ajan kuralı:** Aynı anda tek Unity-süren ajan; aksi hâlde sahne dosyaları çakışır.
- **MCP ile ortam erişimi:** Ajan, sahne nesnesi/SO/sprite işlemleri için ayrı bir insan adımını beklemeyebilir; toplu içerik görevlerinde süreç hızını belirgin artırır.

8.4 Paralel Görev Kuyruğu

Karmaşık işler, birbirine dokunmayan paralel şeritlere bölünür; her şerit bağımsız bir görev belgesiyle tanımlanır ve birleştirme öncesinde cross-review'dan geçer. Bu düzen, bağımsız parçaların eşzamanlı ilerlemesini sağlarken entegrasyon uyumsuzluklarını birleştirme aşamasından önce ayıklar.

8.5 Metodolojinin Değerlendirilmesi ve Sınırları

Test bağımsızlığı: "Kodu yazan yapay zekâ testleri de yazıyorsa bağımsızlık nasıl sağlanır?" sorusuna yanıt şudur: kabul kriterleri, test mantığı ve sınır durumları geliştirici tarafından tanımlanır; ajanlar yalnızca bu insan-tanımlı mantığı koda döker. Görsel QC süreci (Bölüm 9), otomatik testlerin kapsamadığı durumları yakalayan bağımsız bir doğrulama katmanıdır. **İnsan müdahalesi gerektiren alanlar:** tasarım zevki ve oyun hissi hiçbir zaman ajanlara devredilmemiştir; "bu oda sıkıcı mı", "bu knockdown ağır hissediyor mu", "bu güzel mi" kararları yalnızca gerçek oyun oturumundan sonra verilebilmiştir. **Korumalar:** otomatik denetimlerin tek başına yeterli olmadığı durumlar için zaman-damgası doğrulaması ve yazandan bağımsız inceleme gibi süreç kuralları uygulanır.

Araçlar; orchestrator/reviewer ayrımı, karar belgesi zorunluluğu, tek-Unity-ajan kuralı ve cross-review matrisinden oluşan bir mimariyle orkestre edilmiştir. Projenin katkısı yalnızca AI kullanımında değil, AI çıktısını denetlenebilir mühendislik adımlarına dönüştüren süreç tasarımıdır.

Çapraz-denetim örneği. Demo hazırlık aşamasında, bağımsız bir Kalite Kontrol Ajanı danışma sürecine entegre edildi. Bu ajan, diğer danışmanların benimsediği bir öneriyi (koşu içi para birimi göçü) bağımsız kanıtla reddederek demo öncesi kapsam genişlemesinin önüne geçti. Ajanın bulgusu, danışma paneli sentezini değiştirdi ve riskli bir değişiklik uygulanmadan önce elendi. Bu örnek, farklı modellerin çapraz denetimiyle risk tespiti disiplinini somutlaştırmaktadır: çok-danışmanlı süreç yalnızca onay toplamak için değil, gerçek risk tespiti için kullanılmıştır.

8.6 İnsan-YZ İş Bölümü ve Çalışma Prensipleri

Proje boyunca farklı rollere sahip yapay zekâ araçlarıyla iş birliği içinde yürütüldü. Benimsenen yaklaşım, çok-ajanlı yazılım geliştirme (ChatDev [2]), akıl yürütme ile eylemi birleştiren ajan mimarileri (ReAct [8]) ve çok-ajanlı diyalog çerçeveleri (AutoGen [9]) gibi güncel çalışmalarla aynı çizgidedir; bu projedeki ayırt edici yön, orkestrasyon ve kabul kriterlerinin tümüyle tek geliştiricide kalması ve ajanların yalnızca verilen kararları uygulamasıdır.

Orkestrasyon Katmanı

Geliştirme sürecinin tamamında yönlendirme, önceliklendirme ve kabul kriterleri belirleme sorumluluğu geliştiricinin kendisinde kaldı. Yapay zekâ araçları bu kararları almadı; aldıkları kararları değil, onlara verilen kararları uyguladılar.

Tüm koordinasyon, geliştiricinin etkileşim kurduğu terminalde çalışan bir **orkestrasyon katmanı**¹ üzerinden yönetildi. Orkestratörün görevi tek bir büyük işi doğrudan çözmeye çalışmak değildi; işi

anlamalı parçalara bölmek, her parçayı uygun uygulayıcıya yönlendirmek, çıktıları bir araya getirmek ve sonuçları geliştirici onayına sunmaktır.

Ajan Roller

Sistemin üç işlevsel katmanı vardır:

- **Uygulayıcılar** — Kod yazan, dosya değiştiren, doğrulama komutu çalıştıran ajanlardır. Karmaşıklık düzeyine göre iki ayrı Kod Üretim Modülü kullanıldı: çok-dosya değişiklikler için derin bağlam gerektiren model; mekanik ve yinelemeli görevler için daha hafif model¹. Bu modeller arka planda görev kuyruğuna alınarak; biri derin statik analiz, diğeri hızlı bilgi-toplama alanında çalıştırıldı.
- **Denetçiler** — Kalite Kontrol Ajanı, salt-okunur erişimle çalışır; yazan ajandan bağımsız olarak çıktıyı inceler ve doğrulama raporunu doğrudan geliştiriciye iletir. Bu katmanın varlığı, uygulayıcı ile denetçi arasındaki ayrımı kurumsal bir kural olarak hayata geçirir: **yazan denetlemez**.
- **Mimari Değerlendirme Paneli** — Tek bir sistemin ötesine geçen, birden fazla bileşeni aynı anda etkileyen kararlar için üç farklı Mimari Çapraz-Doğrulama Modelinden oluşan bir değerlendirme paneli oluşturuldu. Panel bağımsız bakış açıları üretir; sentezi geliştirici yapar. Bu yöntem, proje boyunca demo kapsam kitleme, ses mimarisi ve görsel kalite kararları gibi geri dönüşü güç seçimlerde uygulandı.

¹ Projede kullanılan modeller: Orkestrasyon Katmanı ve Kalite Kontrol Ajanı rolünde Claude (Anthropic); Kod Üretim Modülü kanalında Codex (OpenAI) ve Claude alt-ajanları; Mimari Çapraz-Doğrulama Modelleri olarak Gemini Pro ve Gemini Flash (Google DeepMind). Model seçimi proje geliştirme döneminin mevcut araçlarını yansıtmaktadır.

Temel Disiplinler

Sistemin işleyebilmesi için aşağıdaki disiplinler proje kuralı olarak belgelendi ve her oturumda uygulandı:

- **İddia ≠ kanıt**: Bir ajanın "başarılı" demesi, işin tamamlandığının kanıtı sayılmadı. Her kritik çıktı veri tabanlı doğrulama (birim testi, ekran görüntüsü, runtime API çağrısı) ile teyit edildi.
- **Tek Unity ajanı**: Aynı anda sahnede değişiklik yapan birden fazla ajan çalıştırılmadı; bu kural, eş zamanlı çalışan iki ajandan birinin sahneyi kendi referansına geri yüklerken diğerrinin değişikliğini silmesi üzerine benimsendi.
- **Oturlararası hafıza**: Yapay zekâ araçları her oturumda geçmiş bağlamı yeniden taramak zorunda kalmadı; tasarım kararları, teknik tercihler ve geçmiş hatalar kalıcı bellek dosyaları ve NotebookLM bilgi tabanı aracılığıyla oturumdan oturuma taşındı. Kod tabanının kendisi ise graphify aracıyla 6925 düğümlü bir bilgi grafiğine dönüştürülerek mimari sorularda ajanların kaynak tüketimini en aza indiren, sorgulanabilir bir hafıza katmanı oluşturuldu.

Mühendislik Çıktısına Yansıması

Bu yaklaşımın asıl değeri, prompt mühendisliği tekniklerinde değil; tek bir geliştiricinin aynı anda birden fazla teknik katmanı yönetebilmesini mümkün kılan yapısal disiplinde yatmaktadır. Oyun mekaniklerini kapsayan C# kod tabanı, oyun içi seviye editörü araç zinciri, piksel-art sprite üretim hattı ve görsel kalite güvence süreci — bunların tamamı paralel olarak ilerledi. Bu ölçeğin tek bir geliştirici tarafından gerçekleştirilmesi, yalnızca araç kullanımıyla değil; araçlar arasındaki iş bölümünü açıkça tanımlayan, her çıktıyı bağımsız denetimden geçiren ve kararları kalıcılaştıran sistematik bir süreç disipliniyle mümkün oldu.

9 DOĞRULAMA: TEST VE KALİTE GÜVENCESİ

9.1 Tek Geliştiricinin Kalite Sorunu

RIMA'nın kalite güvence (QC) süreci üç katmandan oluşur: otomatik birim/entegrasyon testleri, sözleşme (contract) testleri ve sistematik görsel kalite güvencesi. AI'nin kod ürettiği bir bağlamda bu süreç kritiktir: otomatik üretilen bir kod parçası derleniyor olabilir ama sistemin geri kalanıyla beklendiği gibi konuşup konuşmadığı ayrıca doğrulanmalıdır.

9.2 Test Altyapısı

Test paketi Unity Test Runner üzerinde EditMode ve PlayMode testlerinden oluşur. EditMode testleri sahne çalıştırılmadan senkron koşar; PlayMode probe'ları gerçek sahne yüklemesini, bileşen başlatmayı ve çok-kareli asenkron etkileşimleri test eder. Kategoriler: Bootstrap/Wiring, Combat/Encounter, Room/MapDesigner, Props/Composition, RoomPainter/Brush, Movement ve UIFlow.

Ölçüt	Değer
EditMode test dosyası	89
PlayMode test dosyası	12
Bağımsız sözleşme dosyası	4
EditMode test tanımı	508
PlayMode test tanımı	41
Toplam test tanımı	549
Son kayıtlı EditMode koşusu	410 PASS / 0 FAIL / 1 inconclusive
Sayım tarihi	2026-06-19

Tek belirsiz (inconclusive) test, Unity editörü dışında erişilemeyen bir sahneleme bağımlılığını kapsar; işlevsel bir hata değildir. Envanter sayısı (549), kayıtlı koşu tamamlandıktan sonra eklenen test gruplarını (walkable, JSON round-trip, gate-slot, screenshot imza) içerdiğinden koşu toplamından büyüktür.

9.3 Görsel Oda Kalite Güvence Süreci

Otomatik testlerin kapsamadığı bir doğrulama alanı vardır: kod açısından geçerli ama sahnede görsel olarak yanlış görünen durumlar. Bunun için programatik bir görsel QC süreci kurulmuştur: her RoomTemplateSO, _Arena'da IsoRoomBuilder ile yeniden inşa edilir; her inşadan sonra zemin karosu, uçurum sprite ve prop sayısı kaydedilir ve editör görüntüsünden bir ekran görüntüsü alınır.

Sürecin temel ölçütü propOutsideFloor sayacının sıfır olmasıdır: hiçbir prop ada zemininin dışına düşmemeli, uçurum sprite'ları yürünebilir karoların üzerinden geçmemelidir. Bu doğrulama, prop koordinatlarını bake öncesi zemin hücre kümesine karşı kontrol eden bir kuralla desteklenir; ölçütü karşılamayan şablonlar düzeltilir ve süreç sayaç sıfırlanana kadar yeniden çalıştırılır.

Aşağıdaki tablo, üç temsili şablonun doğrulama özetini vermektedir:

Şablon	Zemin Hücresi	İnşa Edilen Prop	Ada Dışı Prop	Sonuç
combat_large_diamond_01	212	4	0	PASS
combatlarge_twin_basins_01	599	1	0	PASS
Treasure_01	42	4	0	PASS

Bu sürecin kalıcı çıktısı, gelecekteki doldurucu geçişlerini de engelleyecek bir editör/üretim-zamanı zemin doğrulama kuralının koda işlenmiş olmasıdır.

9.4 Üçlü Kalite Güvencesi

Katman	Ne yakalar	Örnek
Otomatik test	mantık/sözleşme hataları	gate-slot kuralları, JSON round-trip
Görsel QC	yapısal geçerli ama görsel yanlış	ada dışına taşan prop
Bağımsız inceleme	çalışan ama yanlış entegre kod	i-frame sızıntısı, giriş tamponu regresyonu

Otomatik testler hız ve tekrarlanabilirlik sağlar; PlayMode probe'lar sistem sınırlarındaki hataları (singleton yaşam-döngüsü, sahne-geçiş etkileri) ortaya çıkarır; görsel QC veri-güdümlü içerik üretiminin görsel sapmalarını programatik raporla kanıtlanabilir kılar. Üç katmanın birlikte çalışması, tek-geliştirici + AI üretimi kombinasyonunda kalite güvencesine somut bir model sunar.

10 GRAPHIFY KOD-GRAFI AUDİT

10.1 Yöntem

Projenin "environment + tooling" tezini öznel iddiadan çıkarıp ölçülebilir bir kanıta dönüştürmek için tüm kod tabanı graphify ile bir bilgi grafına dönüştürülmüştür. Araç, C# kaynak dosyalarını parse ederek sınıflar ve bağımlılıklar arası bir grafik kurar, ardından bu grafiği topluluk-tespiti (community detection) ile kümelere ayırır. RIMA kod tabanı için sonuç: **6925 node / 118 community**.

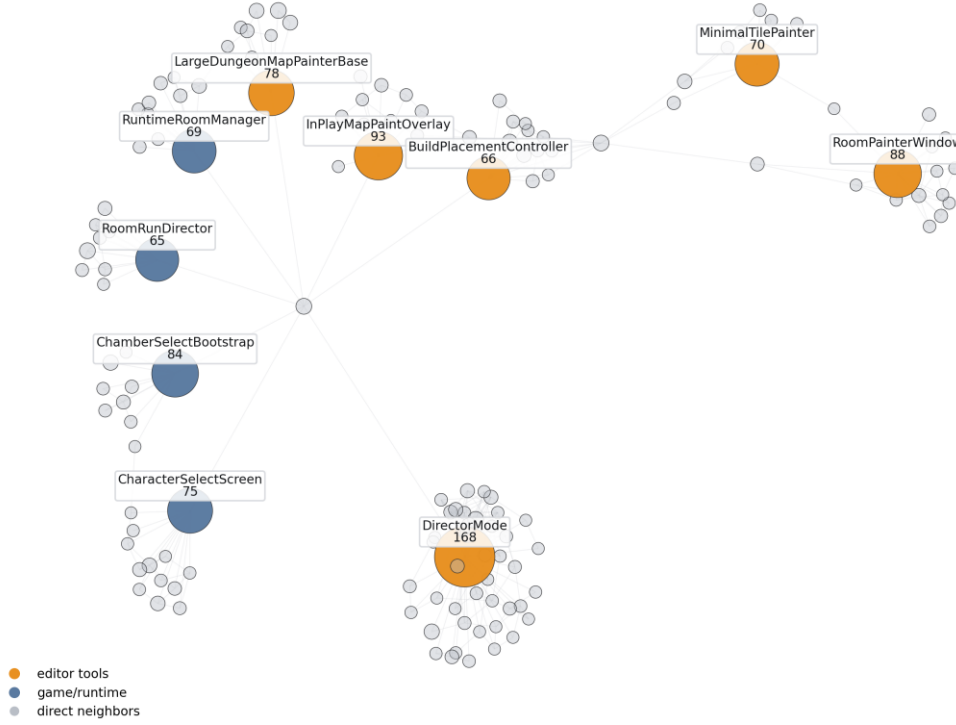
10.2 God-Node Analizi: Araç Katmanı Tezinin Sayısal Kanıtı

Bir grafikte en yüksek bağlantı derecesine sahip node'lar ("god-node"), sistemin etrafında döndüğü merkezlerdir. RIMA'da en bağlı 10 node analiz edildiğinde belirgin bir topolojik dağılım görülür: bunların **6'sı editör/araç (tooling) sınıfıdır**, yalnızca 4'ü oyun/runtime sınıfıdır. Bu, "RIMA aslında bir geliştirme-ortamıdır, üzerine oturan oyun ise bu ortamın bir tüketicisidir" tezinin doğrudan, sayısal kanıtıdır.

En bağlı düğümler Şekil 10'da görselleştirilmiş olup aşağıdaki tabloda özetlenmiştir:

Düğüm	Tür	Rol
DirectorMode (168)	Araç (Tooling)	Çalışma zamanı arayüz fabrikası — en bağlı düğüm
InPlayMapPaintOverlay (93)	Araç (Tooling)	Oyun-içi harita boyama katmanı
RoomPainterWindow (88)	Araç (Tooling)	Editör oda boyama penceresi
ChamberSelectBootstrap (84)	Çalışma zamanı	Karakter seçim önyükleyici
LargeDungeonMapPainterBase (78)	Araç (Tooling)	Harita boyama taban sınıfı
CharacterSelectScreen (75)	Çalışma zamanı	Karakter seçim ekranı
MinimalTilePainter (70)	Araç (Tooling)	Karo boyama aracı
RuntimeRoomManager (69)	Çalışma zamanı	Çalışma-zamanı oda yöneticisi
BuildPlacementController (66)	Araç (Tooling)	Build Mode prop yerleştirme
RoomRunDirector (65)	Çalışma zamanı	Oda yaşam-döngüsü

10 most-connected nodes - 6 are editor tools (orange), 4 are game/runtime (blue)

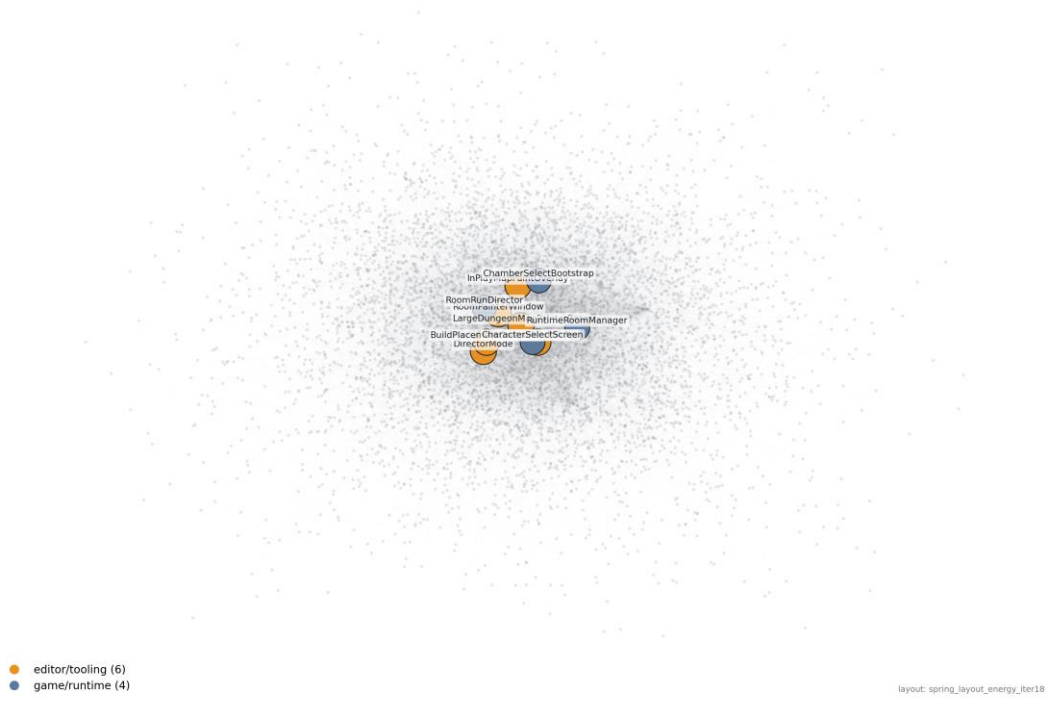


Şekil 10: Graphify — en bağlı 10 node; turuncu = editör araçları (6), mavi = oyun/runtime (4)

10.3 Tam Graf Görünümü

Şekil 11, tam kod grafiğinde 118 topluluğun renk-kodlu kümeler hâlinde dağılımını gösterir; araç sınıfları (Map Designer, Build Mode, Director Mode, boyama araçları) merkezde yoğun bir küme oluştururken oyun sistemleri (Skills, Combat, Enemies) çevresel kümeler hâlinde bu merkeze bağlanır. Bu topolojik ayırım, "araç / çalışma zamanı" katmanlarının birbirinden bağımsız değiştirilebileceğini (bkz. §2.2) ve Bölüm 1.4'te gözlenen "araçlar kod tabanının beşte biri" bulgusunu bağımsız bir metrikle doğrular.

RIMA codebase - 6925 nodes, 14321 edges - 6 of 10 most-connected = editor/tooling



Şekil 11: Graphify tam kod grafiği — 6925 node, 118 community

11 SONUÇ VE GELECEK GELİŞTİRMELER

11.1 Ulaşılan Yer

Projenin başındaki soru — tek geliştirici kapsamlı bir aksiyon-roguelite'i nasıl yapabilir? — iki paralel yanıtla ele alındı: içerik ile sistemi ayıran veri-güdümlü mimari ve bunun üzerinde çalışan çok-ajanlı geliştirme süreci. Bölüm 9 ve 10'da ayrıntılı sunulan sayısal kanıtlara ek olarak, ulaşılan temel çıktılar şöyle özetlenebilir:

- **Oyun sistemi altyapısı:** 10 sınıflık veri altyapısı (her biri farklı kaynak ekonomisi ve oynanış diliyle); 4 sınıfın çalışan kontrolcüsü mevcut, demo'da Warblade ve Elementalist uçtan uca oynanabilir; tam oynanabilir akış (Ana Menü → Karakter Seçimi → Savaş → Yetenek Seçimi → Boss → Meta-ilerleme).
- **Araç ve içerik katmanı:** Oyun-içi araç zinciri (Build Mode, Director Mode, F1, Map Designer, Room Browser, RoomJsonImporter, ScreenshotMode); oyun-hissi katmanı (darbe-duraklaması, CombatJuice, telegraph, 18 CCO SFX + 9 kanal).
- **Graphify mimari kanıtı:** En bağlı 10 düğümün 6'sı araç sınıfı; tooling ve runtime katmanlarının grafikte ayrı bölgelerde konumlandığı görsel olarak doğrulandı (bkz. §10).

Metodoloji açısından, çok-ajanlı geliştirme sürecinin yalnızca "AI kullanmak" olmadığı gösterildi: rol tanımları, görev belgeleri, karar dökümanları, cross-review ve doğrulama-kanıtı gerekliliğinden oluşan bu süreç geliştirici tarafından tasarlandı ve işletildi.

11.2 Yol Haritası

- **Silah üretimi ve sınıf derinleştirme:** Kalan sınıfların sınıfa özel silahları (sprite + mekanik) ve placeholder skill'lerin gerçek implementasyonu en büyük içerik bloğudur.
- **Elementalist büyü VFX:** Particle/shader düzeyinde efekt üretimi; kod ve araç altyapısı hazır, kullanıcı onayıyla başlayacak. Elementalist'in 8 ayrı yön idle klibi üretilmiş ve animasyon denetleyicisine entegre edilmiştir; sınıf gerçek 8-yön ile oynanabilir durumdadır. Kalan kapsam: büyü particle/shader VFX efektleri.
- **Ses ve müzik:** Demo CCO SFX + müzik yatağı yeterlidir; özgün müzik ve sınıfa-özgü prodüksiyon sesleri gelecek kapsamındadır.
- **Boss geliştirme ve Rift portalı entegrasyonu:** Nihai boss "The Architect"ın zengin faz yapısı ve portal sprite'larının tam görsel entegrasyonu; Act 2+ için biyom çeşitliliği.
- **Denge ve playtest turları:** Skill ağırlıkları, tier dağılımı, can değerleri ve oda zorlukları şu an tasarım-temelli başlangıç değerleridir; gerçek oyuncu verisiyle denge turları gerekir.
- **Steam yayın hazırlığı:** Mağaza sayfası, sistem gereksinimleri, erişilebilirlik, Steamworks ve Early Access planlaması.

11.3 Çıkarılan Dersler

- **Mimari kararları erken kilitlemek zaman kazandırır.** Sahne-bazlıdan veri-güdümlüye geçiş geç alınsaydı üretilmiş içeriğin çoğu yeniden yazılırdı.
- **Testler "kod çalışıyor mu" sorusundan fazlasını sormalıdır.** Yapısal testleri geçen birçok oda prop'larını ada dışında bırakıyordu; görsel çıktı içeren projeler görsel doğrulama katmanı gerektirir. Birim testlerinin yeşil geçtiği bir savaş odasının uçtan uca akışta düşmanların boşta kaldığı, çalışmayan bir oda olduğunun ortaya çıkması bunu en net biçimde gösterdi.

- **AI ajanlarına güven teyide dayanmalıdır.** Sessiz başarısızlık birden çok kez yaşandı; geri dönüşü zor işlemlerde ajan raporu bağımsız doğrulanmalıdır.
- **Tasarım zevki ve oyun hissi devredilemez.** Bir odanın sıkıcılığı, bir knockdown'ın ağırlığı, bir skill seçiminin anlamlılığı yalnızca gerçek oyun oturumundan sonra değerlendirilebilir; geliştirici son karar mercii olarak her zaman sürecin içindeydi.

11.4 Değerlendirme

Bu proje; veri-güdümlü mimari, çok-ajanlı geliştirme süreci ve programatik kalite güvencesinin tek-geliştirici ölçeğinde nasıl uygulanabileceğini belgelemektedir. Oynanabilir tam döngü, 10 sınıflık veri altyapısı, 26 oda şablonu, 549 test tanımı envanteri ve graphify ile sayısal olarak kanıtlanan tooling tezi ile ulaşılan kapsam, benimsenen mimari ve metodolojik kararların somut çıktısıdır. Temel mühendislik altyapısı bu çalışmanın kapsamı dahilinde tamamlanmış; ticari kaliteye taşıma yol haritası §11.2'de özetlenmiştir.

11.5 Tasarım Kararları ve Geliştirme İlkeleri

Geliştirme süreci boyunca, gerçek mühendislik problemleriyle karşılaşıldıkça benimsenen ve sonraki tüm çalışmaya yön veren dört temel ilke öne çıkmıştır. Bu ilkeler tek-geliştirici + AI-destekli üretim bağlamında doğruluğu ve öngörülebilirliği sağlayan disiplinlerdir:

- **Görüntülü ve veri-temelli doğrulama.** Bir çıktının kabul edilebilmesi için iddia değil, kanıt aranmıştır; yapısal bir testin yeşil geçmesi, sistemin gerçekten çalıştığının kanıtı sayılmamıştır. Nitekim birim testleri yeşil geçtiği hâlde uçtan uca akışta düşmanların boşa kaldığı bir savaş odasının ancak gerçek koşu doğrulamasıyla ortaya çıkması, bu ilkeyi somutlaştırmıştır.
- **Uçtan uca akışın test edilmesi.** Bileşenlerin tek tek doğru çalışması, bütünün doğru çalıştığı anlamına gelmemiştir; ilerleme istatistikleri, sahne geçişleri ve ödül akışı gibi sistem sınırlarındaki hatalar yalnızca Ana Menü'den boss'a uzanan tam akış sınırlarında görünür hâle gelmiştir.
- **Paylaşılan çalışma-zamanı durumunda tek sahiplik.** Birden fazla bileşenin aynı paylaşılan değişkene yazması öngörülemeyen davranışa yol açtığından, böyle durumlarda yazma yetkisi tek bir sürücüyü verilmiştir (örn. iki efekt sürücüsünün `Time.timeScale`'i aynı anda yazması oyunu donduruyordu; yazma yetkisi tek sürücüyü devredilince çakışma ortadan kalktı).
- **Yazandan bağımsız denetim.** Bir çıktının kalite kontrolü, onu üreten ajan tarafından yapılmamıştır; uygulayıcı ile denetçi rolleri her zaman ayrı tutularak bağımsız bir inceleme katmanı korunmuştur.

EK A — ELEMENTAL DURUM EFEKTLERİ

Elementalist sınıfının Ateş ve Buz yetenekleri, düşmana görsel bir durum boyası uygulayarak oyuncuya düşmanın anlık durumunu doğrudan iletir. Ateş saldırısı hedefi kırmızıya boyar ve zamana yayılı hasar (Burn / yanma) uygular; Buz saldırısı hedefi maviye boyar ve hareket hızını düşürür (Chill / üşüme). Bu renk-kodlu geri bildirim, ayrı bir arayüz göstergesi gerektirmeden durumun doğrudan düşman sprite'ı üzerinden okunmasını sağlar; etki, oyun-içinde varsayılan demo düşmanı üzerinde doğal olarak tetiklenir.

Kaynakça

- [1] Bridson, R. (2007). Fast Poisson disk sampling in arbitrary dimensions. *SIGGRAPH Sketches*, 22.
- [2] Qian, C., Liu, W., Liu, Y., Chen, W., Yao, Y., & Zhou, Z. (2023). ChatDev: Communicative Agents for Software Development. *arXiv preprint arXiv:2307.07924*.
- [3] Shaker, N., Togelius, J., & Nelson, M. J. (Eds.). (2016). *Procedural Content Generation in Games*. Springer.
- [4] Schreiber, I., & Romero, B. (2012). *Game Design Concepts: An Introduction to Game Design and Game Analysis*. CRC Press.
- [5] Supergiant Games. (2021). *Hades: The Design Philosophy Behind It* [GDC Talk]. Game Developers Conference.
- [6] Unity Technologies. (2017). *ScriptableObject Architecture in Practice* [Unite Talk]. Unite Europe 2017.
- [7] Adams, E., & Dormans, J. (2012). *Game Mechanics: Advanced Game Design*. New Riders.
- [8] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*. arXiv:2210.03629.
- [9] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.